BCOM
Barcelona Computational Foundation

BARCELONA COMPUTATIONAL FOUNDATION

Working Paper WP0195

Pillar: Kolmogorov Theory (KT)

A Lean 4 Formalization of Kolmogorov Theory

Axioms, Theorems, and Project Status

Machine-checking the KT corollaries against an explicit AIT interface

Giulio Ruffini with Claude Opus 4.8

Barcelona Computational Foundation · Neuroelectrics, Barcelona

giulio.ruffini@bcom.one

November 2026

https://bcom.one · Lean source: https://github.com/giulioruffini/KTAIT ·
DOI: 10.5281/zenodo.21008868

Abstract

We are building `KTAIT`, a Lean 4 + Mathlib formalization whose purpose is narrow and load-bearing: to *machine-check that the Kolmogorov-Theory (KT) corollaries follow logically from an explicit algorithmic-information-theory (AIT) interface*, and that the KT ontology is typed so that structural errors (e.g. comparing a whole pattern to its own part) cannot silently compile. We do *not* re-prove classical AIT: the standard AIT and probability results (the invariance theorem, the coding theorem, symmetry of information, Kraft's inequality, Bayes' rule, the Solomonoff–Levin universal semimeasure, the recursion theorem, Rice's and Chaitin's theorems, and the Vereshchagin–Vitányi structure-function results) are taken as a clearly delimited *axiom layer*. Every KT corollary is then *proved* from that layer with no `sorry`, and a toy model witnesses that the axiom layer is jointly satisfiable (so the corollaries are not vacuously true). This working paper states the methodology, fixes the axiom layer, inventories every theorem/lemma across the KT corpus with its current formalization status, and sets a prioritized roadmap.

Keywords: Kolmogorov Theory, algorithmic information theory, Lean 4, Mathlib, formal verification, algorithmic regulator theorem, persistence, self-model.

Classification: MSC 68Q30, 03B35, 68V15 / ACM F.4.1

Contents

1 Introduction and methodology **4**
1.1 The one rule 4
1.2 Why this is worth doing 4
1.3 Soundness: named hypotheses, not global axioms 4
1.4 Scope 4
1.5 Citing the formalization 4

2 The standard AIT / probability axiom layer **5**

3 Current status of the Lean development **6**

4 Theorem inventory and formalization status **35**

5 Roadmap **58**

6 Conclusion **58**

1 Introduction and methodology

KT has matured to the point where the dominant risk is no longer in the classical results it invokes, but in whether the *new* KT corollaries are *correctly typed* and *logically entailed* by those results. This project addresses exactly that risk with a proof assistant.[1]

1.1 The one rule

The one rule

Standard AIT (and probability) theorems are allowed to be *axioms*. KT corollaries must be *proved* from those axioms with no `sorry`.

If a KT corollary needs a `sorry`, that is a real gap in the paper. If a classical AIT fact is an axiom, that is honest and expected: we are checking the *dependency structure*, not the AIT foundations.

1.2 Why this is worth doing

The bug class we most want to exclude is *ontological*. For example, writing $\mathrm{I}_K(A : \mathcal{S})$ where $\mathcal{S} \subset A$ is structurally vacuous (a whole versus its own part); the correct object is *temporal*, $\mathrm{I}_K(\mathcal{S}_t : \mathcal{S}_{t+\tau})$ or $K(\mathcal{S}_{t+\tau} \mid \mathcal{S}_t, E, C) \ll K(\mathcal{S}_{t+\tau} \mid C)$. In `KTAIT` the KT roles (*substrate, pattern, self-code, readout, regulator, time*) are *distinct types*; the only bridge to a common carrier is an explicit projection, so the bad version *fails to type-check*. The whole-vs-part error becomes a typed obligation, not a silent mistake.

1.3 Soundness: named hypotheses, not global axioms

A naïve encoding `axiom F : forall (F : AITFrame), P F` asserting a law for *every* frame is *inconsistent* here: one can hand-build a frame violating it (e.g. `slack=0` with mismatched K) and derive `False`, which would make all corollaries vacuously true. We therefore state each AIT fact as a *named hypothesis* (a `Prop` about a frame) and let each corollary consume it. Consequently `#print axioms` on every KT corollary shows only Lean's core axioms (`propext`, `Classical.choice`, `Quot.sound`) — strictly stronger than "rests on named AIT axioms." The toy model (§3) then exhibits a concrete frame satisfying the hypotheses, witnessing joint satisfiability.

1.4 Scope

The current effort is **Level 1** (a typed, axiomatic KT/AIT interface) plus **Level 1.5** (the faithful *probabilistic* ART, including its Bayesian posterior and the coding-theorem bridge). Out of scope for now: a computability-backed K from a concrete universal machine (Level 2), and the differential-geometric (Lie-group / Noether) layer of the world-models paper,[4] which is a separate track.

1.5 Citing the formalization

Papers in the KT corpus cite this development with two macros. `\lean{...}` marks incidental Lean text: a module filename, a definition, one of Lean's core axioms. `\ktait{decl}` is a machine-checked-claim citation: the prose at that point asserts a result and names the KTAIT declaration that proves it, so which claims are machine-checked is a lookup rather than an interpretation — for a reader, and for any tool indexing the corpus. Names inside `\ktait` must be declarations; the guard rejects filenames, paths, and core axioms there. Both macros are enforced: papers registered in `docs/citing-papers.txt` have every cited name resolved against

the current declarations by `scripts/check_sync.sh`, which the pre-push hook and CI run, so renaming or restating a theorem cannot silently break a citing paper's claim. The canonical preamble definition is `\newcommand{\ktait}[1]{\texttt{\small #1}}`.

A paper's claim is about the commit it pins, not about the author's working tree. Where a registered paper links a KTAIT commit as its source, `scripts/check_pins.py` reads that commit and verifies that its tree defines every declaration the paper cites with `\ktait`; the last-registered version of each paper is enforced, and archived versions warn. The check exists because WP0203 cited `ARTExactForm.lean` for three releases under a pin that predates the module, while every name resolved at HEAD.

2 The standard AIT / probability axiom layer

These are the results we *assume*. They are classical and not in doubt; collecting them here fixes precisely what the KT proofs are allowed to lean on. (Per K's directive, starting from these as axioms — and similarly for Bayes — is acceptable and intended.)

Axiom layer A — complexity

Invariance theorem. K is machine-independent up to an additive $O(1)$.
Coding theorem (unconditional). $c_1 2^{-K(x)} \le m(x) \le c_2 2^{-K(x)}$, i.e. $-\log_2 m(x) = K(x) \pm O(1)$.
Coding theorem (conditional). $m(x \mid y) \asymp 2^{-K(x|y)}$.
Kraft–McMillan. $\sum_p 2^{-|p|} \le 1$, hence $\sum_x m(x) \le 1$.
Symmetry of information. $\mathrm{I}_K(x{:}y) = K(x)+K(y)-K(x,y) = K(x)-K(x \mid y^*)+O(\log)$; chain rule $K(x,y) = K(x) + K(y \mid x^*) + O(1)$.
Multiplicity compression. $N_{\le L}(x) \ge 2^r \Rightarrow K(x) \le L - r + O(\log L)$.
Data processing (unconditional). $\mathrm{I}_K(z{:}y) \le \mathrm{I}_K(x{:}y) + K(z \mid x^*) + O(\log)$: computing z from x creates no information about y beyond the cost of the transformation. **Self-information.** $K(x) \le \mathrm{I}_K(x{:}x) + O(\log)$.

Axiom layer B — probability / Bayes

Solomonoff–Levin universal semimeasure. $m(x) = \sum_{U(p)=x} 2^{-|p|}$; conditional $m(R \mid W) \asymp 2^{-K(R|W)}$.
Bayes' rule + deterministic likelihood. prior $P(p) = 2^{-|p|}$, likelihood $P(x \mid p) = \mathbf{1}\{U(p) = x\}$, evidence $P(x) = m(x)$.
Excess-length decay. $\Pr\{|p| \ge K(x) + k \mid x\} \le 2C\, 2^{-k}$.

Axiom layer C — computability / logic

Kleene's second recursion theorem (quines); **Rice's theorem**; **Chaitin's incompleteness** (T proves no $K(x) > n$ for $n > c_T$; K uncomputable); **Vereshchagin–Vitányi**: the Kolmogorov structure function $h_x(\alpha)$ is uncomputable and minimal sufficient statistics are not locatable.

WP0007 consumes three basic forms of these, and `AlgorithmicEmergence` keeps them apart because they are not interchangeable (its module header lists the four refinements): `KUncomputable` (K is not computable), `KThresholdUndecidable` (the predicate $K(x) < b$ is not *decidable* — it is semidecidable, which is the whole content of the discovery barrier), and `KNotApproximable` (no computable f satisfies $K \le f \le K + c$, which does *not* follow formally from the first as stated). `chaitin_certification_ceiling` consumes the Chaitin

bound in the "certified lower bounds are capped" form.

3 Current status of the Lean development

The repository (https://github.com/giulioruffini/KTAIT) builds green on Lean 4.31.0 + Mathlib v4.31.0. Modules:

Module	Content	Key results
`Ontology.lean`	six typed KT roles + part-whole guard	`SelfCode` carries `isSub`
`Basic.lean`	`AITFrame`, `IK`, `NMAI`, `condStar`	symmetry as named `Prop`
`Probability.lean`	prefix machine, Bayes posterior, coding thm	Lemma 1 (bridge)
`ART.lean`	`AITProb`: positive complexity weights; selected-program length and evidence bridges	ART bounds with explicit constants
`Persistence.lean`	`Pers`, `TemporalSelfInfo`	persistence lemmas
`SelfModel.lean`	`DeltaSelf`, Proposition 3	temporal self-model
`SelfModelLimits.lean`	quine floor, diagonalization, Chaitin ceiling	WP0192 Principle 1
`Lawvere.lean`	`WeaklyPointSurjective`, `lawvere_fixed_point`, `fixed_point_free_forbids_surjection`, `cantor_bool`, `kleene_face`, `self_prediction_bar`, `valenceShift_fixed_point_free`, `valence_self_scoring_bar`	WP0236: one diagonal, O1/O2 as two signs, valence bar
`RegulatorSelection.lean`	`ChainRule`	WP0162 Prop. 1 (regulator selection)
`CoarseGraining.lean`	`RegSuff`, structure function	WP0193 Thms A/B
`AlgorithmicEmergence.lean`	counting bound, construction barriers, schedule refinement, fixed-task construction–decision equivalence	WP0007 barriers, inheritance, discussion proposition
`ComputationCores.lean`	deterministic experiments, observability, coordinate overwrites	WP0229 structural comparisons
`ProgramGenerator.lean`	arithmetic expressions, first jets, and state/auxiliary interfaces	WP0215: generator extraction and conditional recovery
`OperationalAlgebra.lean`	generated actions and state quotients	WP0004: checked algebra and explicit finite examples
`OperationalGalois.lean`	program orbits and contextual congruences	WP0004: checked algebra and explicit finite examples
`OperationalFinite.lean`	finite actions and linear representations	WP0004: checked algebra and explicit finite examples
`OperationalPrograms.lean`	multiplication programs and storage	WP0004: checked algebra and explicit finite examples
`OperationalApproximation.lean`	composition of approximate reductions	WP0004: checked algebra and explicit finite examples
`OperationalReversibility.lean`	history-based reversible simulation	WP0004: effective computation and reversible actions
`OperationalUniversality.lean`	a fixed universal effective action	WP0004: effective computation and reversible actions

`OperationalActions.lean`	action kernels and faithful representations	WP0004: realized transformations and marked actions
`AnswerTableCoding.lean`	partial-recursive answer prefixes and uniform coding	WP0215: checked deductions and explicit premises
`WholeFunctionTree.lean`	threshold trees and unrestricted whole-function multiplicity	WP0215: checked deductions and explicit premises
`WholeFunctionPrograms.lean`	certified whole-function source reconstruction	WP0215: checked deductions and explicit premises
`ClosureExtensions.lean`	finite stochastic quotients and ordered grain families	WP0215: checked deductions and explicit premises
`ClosureInvariance.lean`	general state maps, within-fiber permutations, and group quotients	WP0215: checked deductions and explicit premises
`FiniteTableCode.lean`	literal binary tables and fixed-interpreter cost	WP0215: checked deductions and explicit premises
`NearMinimalPrograms.lean`	staged source enumeration and near-minimal reconstruction	WP0215: checked deductions and explicit premises
`PredictiveThermostat.lean`	saturated forecast controller and approximation error	WP0215: checked deductions and explicit premises
`ClosureSymmetry.lean`	symmetry quotients, reversibility, and what a symmetry removes	WP0255: checked deductions and explicit premises
`SymmetryCompression.lean`	orbit-table reconstruction; represented quotients; conditional coding premises	WP0215: equivariant-law counting, closure, and code costs
`OperationalInformation.lean`	partial-recursive search; binary relative entropy; named coding, Meyer, and quantum-budget premises	WP0251: replacement complexity, report bounds, and threshold margins
`FunctionalCore.lean`	exact and approximate task descriptions	WP0007 minimum-description bound
`Certification.lean`	partial semantics; computable certificate stream	WP0228 unrestricted minimum
`Marking.lean`	persistent finite marking; event witnesses	WP0228 budget and eventual coverage
`Encoding.lean`	gamma-style headers; fixed-width indices	WP0228 parsing and prefix-freeness
`Counting.lean`	binary universe; index width; scalar algebra	WP0228 finite counting core
`Contrast.lean`	contrast fibre posterior	WP0162 Q3 (Eq. 26)
`ShiftInvariance.lean`	`CheapTransform`, `PairSymmetric`	WP0202 App. A (the null-hypothesis error)
`PushPull.lean`	`Plant`, `Admissible`, `InwardOnBoundary`	WP0186 Thms 1–2, Cor. 1, gap Lemma 1
`ModelOrPay.lean`	`MAICapped`; multiplicity witnesses	WP0203 Props. 1–2
`ReversibleReconstruction.lean`	`ConditionalMapBound`	WP0203 v20: inverse joint dynamics, world decoding, and null replay
`GenerativeRegulation.lean`	retained model, additive compensation, map-cost bounds	WP0203 v22: lossless model-based regulation
`ActionLog.lean`	`trajectory`; decoder identity; log, preload and gap cost bounds	WP0203 Round 1: the thermostat with an action log
`ResidualTransfer.lean`	nonnegative weights; positive finite evidence mass	WP0203: evidence-local confidence and residual-tail transfers

ARTExactForm.lean	Chain, CondChain, OutputsComputable; pair/-context conversion costs	WP0203: exact exponents, selected-length and symmetric-information bridges
IndependentRegulation.lean	coding domination, pair Kraft, normalized product weights	WP0203: local GART thresholds and residual-conditioned sampling
RegulationBalance.lean	conditional information, reconstruction, incomplete records	WP0203 v23, the regulation balance
GroundedRegulation.lean	Grounded, residual, IKCeiling, PairMono	WP0203 grounded exposition, repaired corollary, self-regulation/implicit-self-model corollary
HorizonConditioning.lean	conditioning_on_horizon, horizon families	WP0203 v32.10: GART read across horizons; horizon-free shared information
InterventionKnowledge.lean	SelfInformation, DataProcessingSnd, ChainThroughOutput, ChainPair	WP0203 v34.3: descriptor bound, descriptive saving, shared-content split
OrbitLabel.lean	conserved orbit labels	WP0162 App. C
NoetherFlow.lean	trajLabel, flow symmetries	generalized Noether (abstract core)
Decoder.lean	Achievable, Inverts, dovetail	WP0058 Prop. 1(i)–(iii), axiom-free
WriteBack.lean	bandwidth, DataProcessingCond	WP0058 Prop. 2–3, channel bound
Retention.lean	J, qStar, FirstCrossing	WP0058 Prop. 4 (optimal retention kernel)
CommonSemantics.lean	DataProcessingIK, SelfInfo (in KTAIT/IS/)	WP0215 common-semantics bound
Boundary.lean	Machine, Reach, InternalIso (in KTAIT/IS/)	WP0215 boundary vs. organization
Unfolding.lean	Recurrent, Layered (in KTAIT/IS/)	WP0215 finite-horizon unfolding
ToyModel.lean	Toy, ToyMachine, ToyGap	satisfiability witnesses
PatternPersist.lean	PP! ontology on AITFrame	agenthood per cell, not automatic; Prop. 3 inherited
PersistenceFlow.lean	flowTuple, FlowSplit5, CIKCeiling, ReconstructionBound, noGapFrame	APB suite: persistence balance, flow corollaries, self-code overload, no-gap witness (roadmap v2)
CompletionLedger.lean	ctxAfter, orderedInfo, orderedCost, OrderedCeilings, OrderedSplit	WP0218 revision: reconstruction and capacity with neutral names, grouped and ordered forms; the five-record APB as the ordered instance
StateGraph.lean	UniqueSucc, UniquePred, IsPermGraph, Steps	WP0218 Appendix E: singleton support layers are the graph of a permutation; exact-step layers are singletons; the orbit label is conserved along allowed transitions

`ModelAccounting.lean`	conditional recoding, acquired model and residual costs; repeated-block records	WP0218 revision: compact accommodation and incompressible innovations, conditional AIT consequences and finite recovery checks
`Localization.lean`	`localization_left/mutual/right`, `CondDataProcessing`	WP0218 localization algebra, reversible balance, recoverable-description overlap, persistence bound
`TokenTransport.lean`	`Mode`, `LocalizationProfile`, `LocalizationPlan`, `nwPlan`, `natAbsDiff`	WP0218 canonical token transport: gates, plans, minimum moved mass (finite, Mathlib-free)
`Agentoptosis.lean`	`AgentoptoticEvent`, bearer/target split	WP0207 v0.8 ontology; deletion threshold
`BadStatements.lean`	guards bite	part-whole; y vs y^*; WP0162 ontology

Proved KT corollaries (no `sorry`; `#print axioms` = Lean core only):

- In `OperationalInformation`, `recover_partrec` and `recover_correct` formalize finite-stage search and correctness of recognized answer tables. `replacement_bound`, `computable_of_bounded_replacements`, `noncomputable_unbounded_replacements`, and `budget_transfer` derive WP0251's conditional AIT bounds from an effective table observer, adequate replacements, and the named standard hypotheses `DecoderCoding` and `Meyer`. Replacement selection need not be computable. `wrong_estimate_below_half` and `correct_estimate_above_half` verify the real-valued decision margins. The construction of the block observer from probability approximators, quantum effects and contraction, matrix nets, copy-count bounds, and biased-source sampling remain paper-level arguments. The separate Lean 4.35 project proves Meyer from an optimal-machine definition; this Lean 4.31 module does not import that proof.
- In `OperationalInformation`, `reliable_report_bound` checks the scalar implication from a reference-surprisal coding inequality and a success-weighted physical budget to a bound on the complexity of a complete reported string. Both inequalities are named premises. Gibbs-state construction, quantum relative-entropy data processing, and the cavity calculation remain paper proofs in WP0251's separate research note; they are not verified by this declaration.
- In `OperationalInformation`, `binaryRelativeEntropy_eq_binEntropy` proves the binary-divergence identity for actual probabilities in $[0,1]$ and reference probabilities in $(0,1)$. `binary_surprisal_le` and `binary_surprisal_le_add_log_two` prove the classical success-weighted surprisal bound, first with binary entropy and then with its upper bound $\ln 2$. These are standard information inequalities. `reliable_report_of_binary_budget` composes that proof with named coding and binary-divergence budget premises; `reliable_report_of_reference_one` handles unit reference probability without evaluating a divergent relative entropy through Lean's totalized logarithm. Quantum data processing, Gibbs states, reference semimeasure construction, uniform coding constants, and the spectral limit remain paper-level obligations.
- In `OperationalAlgebra`, Generator preservation induces the unique surjective homomorphism between realized transformation monoids. Future-observation equivalence is the greatest stable content-preserving equivalence; finite refinement stabilizes before the number of states. Common bijective factors conjugate actions. Surjective factors, projected inverses, prepared reset, and the two-sided shift distinguish global reduction from initialized simulation. Integer

iteration has the usual quotient by its kernel. Checked declarations: `generated_factor_commutes`, `generated_factor_surjective`, `generated_factor_unique`, `common_factor_conjugacy`, `stable_under_words`, `future_equivalence_content`, `future_equivalence_stable`, `future_equivalence_greatest`, `richer_descriptors_refine`, `more_operations_refine`, `refine_le`, `refine_eq_iff_stable`, `refinement_bounds`, `stable_refinement_eq_future`, `proper_refinement_increases_blocks`, `refinement_stabilizes`, `finite_refinement_computes_future`, `refinement_entry_budget`, `surjective_factor`, `finite_surjective_factor_bijective`, `descended_inverse`, `prepared_swap_reset`, `reset_not_global_factor`, `one_sided_shift_factor`, `faithful_integer_iteration`, `composed_factors`, `singleton_content_constant`, `finite_submonoid_inverse`, `involution_generated_elements`.

- In `OperationalGalois`, The orbit and class-preserver maps form an adjunction, recover equivalences, and identify the fixed subgroups of orbit saturation. Structural extractors and forgetting maps order these comparisons. Generated contextual congruence is adjoint to rule extraction; the contextual kernel is the greatest congruence below an extractor kernel. The cyclic-three and modulo-four examples prove the need for subgroup saturation and contextual closure, including strict recovery failure for a restricted ambient group. Checked declarations: `mem_functionGroup`, `orbitRel_apply`, `orbitRel_mono`, `orbitRel_le_ker`, `mem_classGroup`, `classGroup_le_functionGroup`, `mem_classGroup_of_le_ker`, `classGroup_mono`, `orbitRel_le_iff_le_classGroup`, `swap_mem_classGroup`, `orbitRel_classGroup`, `le_saturate`, `saturate_mono`, `saturate_saturate`, `saturate_classGroup`, `fixed_eq_classGroup`, `fixed_le_iff`, `orbitRel_le_iff_le_relative`, `orbitRel_relative_le`, `structuralGroup_eq_classGroup`, `ker_pair_le_ker`, `forget_le`, `orbitRel_le_ker_of_invariant`, `orbitRel_eq_ker_iff`, `generated_isCongruence`, `subset_generated`, `generated_le`, `isCongruence_bot`, `isCongruence_top`, `generated_empty`, `generated_le_iff`, `generated_rules_generated`, `rules_generated_rules`, `generated_le_ker`, `contextualKer_isCongruence`, `contextualKer_le_ker`, `le_contextualKer`, `contextualKer_greatest`, `top_not_generated_bool`, `negPerm_add`, `negPerm_mul_self`, `mem_inversionGroup_iff`, `orbitRel_inversion_iff`, `inversion_orbits`, `orbitRel_one_three`, `not_orbitRel_two_zero`, `addOne_mem_translations`, `inversion_not_congruence`, `inversion_not_congruence_translations`, `parity_iff`, `parity_isCongruence`, `generated_inversion_eq_parity`, `contextualKer_inversion_eq_bot`, `cyclicThree_le`, `orbitRel_cyclicThree`, `saturate_cyclicThree`, `card_cyclicThree`, `card_symmetricThree`, `card_saturate_cyclicThree`, `cyclicThree_ne_top`, `swapGroup_le`, `swapGroup_fix_two`, `relative_recovery_strict`, `cyclicThree_not_fixed`.
- In `OperationalFinite`, A stable finite enumeration is proved equal to the generated transformation monoid. Kernel reduction verifies the four bit transformations, the sixteen-map record/receiver monoid, its twenty-four-map extension, the changed receiver quotient under use bypass, and the counter and Boolean commutator examples. Deterministic matrices preserve composition and distinguish transformations, while the bit example proves that linear extension need not be injective. Checked declarations: `stable_stage_exact`, `stable_stage_card`, `bit_monoid_card`, `bit_relations`, `bit_bijections`, `bit_composition_table`, `bit_all_transformations`, `record_monoid_card`, `extended_monoid_card`, `extension_preserves_generators`, `record_receiver_closure`, `receiver_reversibility`, `receiver_observations_separate`, `receiver_future_eq`, `bypass_future_eq`, `record_future_eq`, `controlled_boolean_involution`, `controlled_boolean_readout`, `controlled_boolean_nonidentity`, `controlled_boolean_two_elements`, `receiver_recoding_nontrivial`, `operation_conjugacy`, `counter_parity_and_carry`, `counter_and_independent_orders`, `parity_counter_order`, `distinct_order_two_actions`, `commutator_and`, `nonidentity_can_fix_readout`, `regular_action_detects`, `action_matrix_composition`, `action_matrix_injective`, `action_`

algebra_basis, bit_matrix_dependence, bit_algebra_not_injective, affine_bit_recoding, affine_bit_encoding.

- In OperationalPrograms, An explicit mutable-register evaluator verifies six programs computing the fifteenth power. Write-version extraction gives two nonisomorphic marked multiplication graphs. A single bijective register renaming gives exactly four storage classes. The four class-preserving permutation groups have orders one, four, thirty-six, and seven hundred twenty. Checked declarations: six_program_denotations, six_distinct_programs, all_permutations_preserve_function, extracted_graphs, distinct_multiplication_graphs, graph_iso_refl, graph_iso_symm, graph_classification, storage_classification, hierarchy_group_orders.
- In OperationalApproximation, A residual estimate and a Lipschitz estimate give the error recurrence for trajectories with the same operation schedule. Iteration proves the geometric, nonexpansive, contractive, and varying-constant bounds. Distance is preserved by isometries, while a three-point example shows that a fixed tolerance is not transitive. Checked declarations: affine_error_bound, nonexpansive_error_bound, contractive_error_bound, operational_error_step, common_schedule_error_bound, lipschitz_schedule_bound, varying_error_bound, tolerance_not_transitive, transported_distance.
- In OperationalReversibility, A transition becomes a permutation by storing previous states and pairing the resulting injective edges. Two involutions give the forward step. Every positive-time state and observation is preserved, the history length records elapsed time, and integer powers act faithfully. The realized cyclic action group is isomorphic to the integers. Primitive recursiveness is preserved in both directions. Checked declarations: record_injective, cross_involutive, turn_involutive, turn_sq, cross_sq, advance_positive, advance_iterate_positive, record_iterate_state, record_iterate_length, simulates, simulation_history_length, observation_simulates, advance_mem, advance_pow_mem, advance_powers_injective, advance_integer_powers_injective, primrec_record, primrec_turn, primrec_cross, primrec_advance, primrec_advance_inverse, primrec_prepare.
- In OperationalUniversality, The history lift preserves the complete partial execution protocol for arbitrary preparation and observation. A fixed primitive-recursive transition runs the bounded partial-recursive-code evaluator and stores its result. Its reversible history extension computes exactly the partial-recursive functions with explicit program preparation and readout. A computable transition with computable preparation and observation has partial-recursive stopping semantics. The ordinary transition is noninjective; the reversible transition and inverse are primitive recursive. No machine-step complexity bound is asserted. Checked declarations: execute_reversible_eq, execute_partrec, step_primrec, initial_primrec, observe_primrec, iterate_step, monoid_evaluates, step_not_injective, universal_step_primrec, universal_inverse_primrec, reversible_initial_primrec, reversible_observe_primrec, reversible_observation, group_evaluates, universal_group, universal_monoid, group_computability_iff, monoid_computability_iff.
- In OperationalActions, An action factors through its faithful realized image. The group quotient removes the action kernel; the monoid quotient removes the kernel congruence. Nonisomorphic cyclic groups of orders four and two realize the same bit transformations. Two faithful order-two actions on four states have different fixed-state counts and admit no action isomorphism, even allowing a group isomorphism. Labeled conjugacy preserving input, completion, and readout transports every finite run. Checked declarations: apply_eq_iff_inv_mul_mem_ker, apply_pointwise_eq_iff, range_faithful, quotientAction_mk,

quotientAction_injective, quotientAction_ker, quotientAction_range, flip_mul_self, c4Action_apply, c4Action_range, c2Action_range, c4Action_range_eq_c2Action_range, c4Action_ker_iff, c4Action_ker_nontrivial, c4Action_ker_ne_bot, c2Action_injective, c2Action_ker, c4_not_isomorphic_c2, fixedCount_conj, transpositionAction_injective, doubleTranspositionAction_injective, transpositionAction_gen, doubleTranspositionAction_gen, transposition_fixedCount, doubleTransposition_fixedCount, c2_elements, transposition_not_intertwines_double, run_conj, marked_run_conj, readout_conj, completion_conj, kernelCon_iff, kernelCon_iff_pointwise, mrange_faithful, quotientMonoidAction_mk, quotientMonoidAction_injective, quotientMonoidAction_mrange.

- In AnswerTableCoding, A partial-recursive evaluator generates finite answer tables. Totality is required only at the selected model code. The general conditional-map coding principle gives one description overhead across models, response functions, and prefix lengths. Checked declarations: collect_partrec, collect_correct, realized_task_computable, answers_length, answer_table_computable, answer_prefix_complexity, total_evaluator_complexity.

- In WholeFunctionTree, Source streams with computable stages generate a threshold tree. Disjoint contributor sets bound every finite antichain, and every prefix of the total target enters. The standard bounded-width tree-labeling principle and fixed compilation law give the whole-function multiplicity bound. The tree-labeling theorem is a named AIT premise; it is not re-proved here. Checked declarations: framed_prefix_mono, framed_prefix_length, admitted_prefix_closed, incompatible_contributors_disjoint, antichain_source_budget, admitted_antichain_width, target_prefix_admitted, parse_frame_append, parse_frame_sound, parse_output_append_stable, parse_output_framed_prefix, parse_output_stream_sound, tree_decode_at_stable, tree_decode_mem_iff, label_produces_decode_complete, label_produces_decode_sound, label_produces_decoder_correct, total_tree_description_bound, whole_function_log_count, whole_function_multiplicity_bound, whole_function_real_log_bound.

- In WholeFunctionPrograms, An executable scan over binary sources and a computable certificate stream produces permanent marks and chronological events. Coverage and event budgets justify the global and local prefix descriptors. Named fixed-compiler laws turn their proved decoding behavior into the certified whole-function reconstruction bound, including partial functions with their domains. Checked declarations: global_event_budget, global_coverage, local_complete, global_decode_stable, local_decode_stable, global_decode_unique, local_decode_unique, global_decoder_correct, local_decoder_correct, global_cost_bound, local_cost_bound, certified_reconstruction, certified_near_minimal.

- In ClosureExtensions, Strong lumpability gives a unique stochastic quotient whose evolution commutes with projection for every initial law and every finite iterate. The content kernel is the greatest unrestricted content-preserving equivalence; declared admissible families can have no coarsest or maximal member. The counterexamples concern abstract equivalence families, not physical admissibility. Checked declarations: lumpable_iff_unique_quotient, quotient_stochastic, stochastic_quotient_exists_unique, pushforward_evolve, pushforward_evolve_iterates, lumpable_linear_combination, content_kernel_is_greatest, greatest_content_equivalence_unique, finite_family_without_coarsest, infinite_family_without_maximal.

- In ClosureInvariance, Closure is equivalent to representative invariance and to a nondecreasing co-membership indicator on paired states. Two-sided closure is equivalent to conservation of that indicator and normalizes the fiber-permutation group. Arbitrary

equivariant group actions induce closed orbit quotients. The module also proves content factorization, operation composition, and finite iteration. Checked declarations: `factor_iff_fiber_constant`, `closure_iff_fiber_constant`, `fiber_constant_iff_representative_invariant`, `unique_reduced_iff_representative_invariant`, `closure_iff_representative_invariant`, `closure_iff_conjugate_subset`, `closure_both_iff_normalizes`, `flow_closure_iff_normalizes`, `Closed.preserves`, `closed_composition`, `reduced_composition`, `reduced_iterates`, `closed_iterates`, `same_macrostate_iff`, `equivalence_indicator_conserved`, `closed_iff_indicator_nondecreasing`, `closed_both_iff_indicator_conserved`, `same_macrostate_iterate_iff`, `flow_equivalence_indicator_conserved`, `equivariant_preserves_orbits`, `symmetry_quotient`, `orbit_update_commutes`.

- In `FiniteTableCode`, The explicit encoder and parser recover finite tables with exactly $k\lceil \log_2 n \rceil$ bits and conditional prefix-freeness. Together with the supplied orbit presentation, named fixed-interpreter and orbit-decoder laws give the equivariant-update complexity bound. The prefix code is proved; an optimal universal-machine construction remains a standard AIT premise. Checked declarations: `encode_length`, `parse_encode_append`, `parse_encode`, `encode_prefix_free`, `state_index_fits`, `literal_table_complexity`, `table_values_length`, `table_values_lt`, `finite_table_coding_bound`, `equivariant_update_code_bound`.

- In `NearMinimalPrograms`, Conditional coding mass bounds the actual finite family of near-minimal programs. A staged enumeration and parsed deficit/index descriptor recover either source from the other, giving an explicit excess-plus-logarithm bound. Coding and universal simulation are named standard AIT premises; interpreter soundness, completeness, and stability are explicit. The module also proves normalized common-semantics and report-information consequences. Checked declarations: `mem_nearFamily`, `near_family_card`, `discoveries_complete`, `near_code_length`, `near_code_prefix_free`, `decodeAt_stable`, `decodeAt_unique`, `near_decoder_correct`, `near_minimal_reconstruction`, `near_minimal_programs`, `normalized_overlap`, `normalized_common_semantics`, `report_fraction_reconstruction`.

- In `PredictiveThermostat`, The actual clipped heating controller contracts in the temperature/forecast-error norm. Finite-horizon perturbation bounds, affine record recoding, and mismatch identities follow. These are real-algebraic controller results; experiential assignments and physical implementations are outside their statements. Checked declarations: `clip_secant`, `forecast_error_step`, `synchronized_step`, `thermostat_contraction`, `stateDist_triangle`, `error_recurrence_bound`, `thermostat_perturbation_bound`, `additive_error_distance`, `thermostat_additive_perturbation_bound`, `decode_recode`, `recoding_conjugacy`, `forecast_error_mismatch`, `clip_bounds`, `command_unique_maximizer`, `normal_gates_step`, `outward_gate_independence`, `gated_square_invariant`, `recode_decode`, `recoding_record_increment`, `synchronized_trajectory`, `numerical_record_commands`, `numerical_record_states`, `temperature_projection_not_closed`, `numerical_read_bypass`, `numerical_write_block`.

- In `ClosureSymmetry`, for an onto state map, conservation of the co-membership indicator is equivalent to forward closure with an injective reduced update, with no invertibility of the microscopic update; for a bijective update it is equivalent to closure of the update and of its inverse, and the reduced update is then a bijection that reduces the inverse. A group action commuting with a bijection commutes with its inverse, so its orbit quotient is closed in both directions, its orbit indicator is conserved, and its reduced update is injective. A symmetry fixing every macrostate induces the identity; a bijection carries fibers onto fibers exactly when it normalizes the relabeling group, and a commuting symmetry then descends to a map

commuting with the reduced update. An orbit of a finite group has at most as many points as the group; a commuting update preserves the fixed-point set. For the trivial partition, a transitive group of bijections commuting with the update forces equal return-time sets (necessity only; the converse construction and the return-map centralizer characterization are not formalized). Checked declarations: `indicator_conserved_iff_closed_injective`, `closed_both_iff_closed_injective`, `reduced_bijective_of_closed_both`, `trivial_partition_closed`, `fiberPerm_induces_id`, `maps_fibers_iff_normalizes`, `descended_commutes`, `orbit_mk_surjective`, `orbit_mk_eq_iff`, `equivariant_symm`, `symmetry_quotient_both`, `symmetry_quotient_indicator_conserved`, `symmetry_quotient_reduced_injective`, `card_orbit_le_card_group`, `fixedPoints_invariant`, `zpow_apply_comm`, `returnTimes_eq_of_commute`, `returnTimes_const_of_transitive`.

- In `SymmetryCompression`, `orbit_table_recovery` and `orbit_table_injective` reconstruct an equivariant update from one image per represented orbit. `equivariant_update_count` bounds the number of such updates by n^k for n states and k orbits. `quotient_update_commutes` and `quotient_update_iterates` prove exact closure and its finite iterates. `equivariant_update_complexity` derives the conditional bound $k\ell + c_{\text{code}} + c_{\text{dec}}$ from the named premises `TableCodingBound` and `OrbitDecoderBound`. Orbit representatives and reconstruction coordinates are supplied; no effective selector or optimal prefix-machine compiler is constructed. The paper instantiates $\ell = \lceil \log_2 n \rceil$ by ordinary finite-table coding and charges any presentation absent from the conditioning context.

- `ProgramGenerator.jet_value`, `jet_derivative`, `symbolic_jet_correct`, and `symbolic_generator_sound` — WP0215's worked example: a recursive syntax-to-syntax transformation extracts the value and derivative at zero in a declared step parameter for finite rational arithmetic expressions over real states. `interface_coefficients` represents separate state and auxiliary coordinates; `scalar_recovery_limit` and `recovery_with_auxiliary_inputs` derive the difference-quotient limit from zero-step identity and auxiliary-independent first jets. `recovered_vector_field_unique` gives uniqueness under those conditions. `generator_unique` proves uniqueness of the extracted derivative; `decay_common_core` verifies the common first jet of explicit Euler and RK4 expressions for scalar decay. These are standard differentiation results applied to the program class, without AIT assumptions. Compiler correctness, LLVM parsing, floating-point rounding, polynomial normalization, state-interface discovery, and experiential sufficiency are not formalized.

- `PP.token_is_agent`, `PP.class_not_agent_here`, `PP.kind_not_agent_here`, `PP.collective_can_be_agent`, `PP.collective_not_agent_here` — the four cells of the WP0207 square (token a_i, constituent class $\mathcal{T}$, collective M, collective kind $\mathcal{L}(M)$) are all *patterns*; agenthood is decided cell by cell on regulatory evidence and follows from patternhood for none of them. The witnesses exhibit a token that is a genuine agent, recurring kinds that are targets without being agents, and a collective shown both ways — composition settles nothing. Possible because agenthood is a *predicate* on the single carrier `Pattern`, following WP0162's "no agent over and above the pattern"; the per-cell reading is WP0207 v0.8's, after the PP! synchronization review.

- `PP.other_target_not_self_targeting` (general, definitional), `PP.AgentoptoticEvent`, `PP.event_agent_not_self_targeting`, `PP.targeted_class_not_agent`, `PP.composition_does_not_imply_agenthood`, `PP.CollectiveSelfTargeting` with its projection lemmas `PP.collective_self_targeting_requires_agent` and `PP.collective_self_targeting_requires_target`, and `PP.terminal_action_underdetermines_target` — the WP0207 v0.8 ontology in `Agentoptosis.lean`: an agentoptotic event requires constituent agenthood and internal selection of the terminal action,

and deliberately carries no target or beneficiary field; an other-directed objective is not telehomeostatic for its bearer; naming a pattern as target promotes nothing; higher-scale telehomeostasis is by definition collective agenthood plus a self-targeting collective objective; and the same terminal action arises under objectives with distinct targets, so self-termination alone cannot identify the persistence target.

- `PP.ordinary_retains_exceptional_terminates` — WP0207's nested-mutualism reading as a state-dependence witness: a single class-targeting objective retains the bearer in the ordinary state and terminates it in the exceptional one, so higher-scale targeting is compatible with keeping the constituent alive almost everywhere and agentoptosis is the reversal region. The mutualistic baseline itself is architectural and evolutionary; no generic mutual-benefit theorem is stated.

- `PP.deletion_threshold` — WP0207's boxed decision rule as elementary algebra over $\mathbb{Q}$: for positive costs, $0 < p\,C_{\mathrm{FN}} - (1-p)\,C_{\mathrm{FP}}$ exactly when $p > C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$. The biological and evolutionary content of WP0207 (Ψ, ΔJ, χ, the filter operator, release, enforceability) is deliberately not formalized.

- `PP.token_not_self_targeting`, `PP.class_is_self_targeting` — telehomeostasis is fixed by the objective alone (WP0162 §3), so a token whose objective proxies its class is a genuine agent that is not telehomeostatic. `PP.agentoptosis_gap` states the conjunction: that gap is the room agentoptosis occupies (WP0207). `PP.token_usually_self_targeting` records the default case: when the objective targets the token itself, it *is* telehomeostatic, so the divergence is the exception and not the rule.

- `PP.collective_not_self_targeting` — the same divergence one grain up: a collective instance whose objective proxies its lineage, with `PP.kind_is_self_targeting` closing the square. This is the snowflake-yeast sign anomaly as a typing fact rather than an exception.

- `PP.persists_iff_nmai` — the module's persistence predicate unfolds definitionally to WP0162's $\mathrm{NMAI}(S_t, S_{t+\tau})$. `PP.Persists` is `Persistence.Persistent` on the pattern's trajectory, so persistence is inherited from `Persistence.lean` rather than restated: rational-valued, normalized, and lag-indexed.

- `PP.agent_has_temporal_self_model` — because `PP.IsAgent` is a positive self-regulation gap Δ_{self} (`SelfModel.DeltaSelf`), Proposition 3 applies to any agent verbatim: its future self-code is cheap given its present organization. Nothing is re-proved; the same named hypotheses are consumed.

- `PP.externally_regulated_not_agent`, `PP.agent_need_not_be_self_targeting` — agenthood is computed from Δ_{self}, never asserted, so a pattern with no gap is refused agenthood; and an agent whose objective targets another pattern is not telehomeostatic.

- `PP.room_not_agent`, `PP.decoupled_not_agent` — locus is *derived* from a directional ablation pair, never asserted, so the externally regulated thermostat room and the decoupled pattern are refused agenthood by computation.

- `AITProb.probabilistic_regulator_theorem` — ART Theorem 2, $\Pr((W,R) \mid x, E) \le C\,2^{M(W:R)}2^{-\Delta}$.

- `AITProb.wrapper_bound` — ART Eq. (6), now *derived* from the coding theorem: $\mathrm{post}(e,x) \le (1/c_1)2^{K(x)-K(e)}$.

- `AITProb.probabilistic_regulator_theorem_sharp` — ART Theorem 2 *sharp form* retaining the regulator cost: $\mathrm{post} \le C\,2^{M(W:R)}2^{-\Delta}2^{-K(R)}$ (no extra hypothesis; the headline drops

$2^{-K(R)} \leq 1$ via `probabilistic_regulator_theorem_of_sharp`).

- `AITProb.theorem1_posterior_tilt` — ART Theorem 1, two-sided posterior tilt.
- `AITProb.theorem3_onoff_evidence` — ART Theorem 3, ON/OFF evidence ratio $\asymp 2^{\Delta}$ (log-free form).
- In `ART.lean`, `AITProb.post` is the abstract weight $2^{-K(e)}/m(x)$. `compatible_program_factor` identifies a compatible selected program's posterior with $2^{-\delta}\mathrm{post}(e,x)$ when its length is $K(e)+\delta$ and its evidence denominator agrees with the frame. `specified_weight_coding_bounds` retains the actual program length and both coding constants. `sharp_art_specified` gives the sharp regulator bound with explicit prefactor $(1/c_1)2^{c_0-\delta}$; `sharp_art_uniform` chooses one upper prefactor before quantifying over candidates, using a common lower bound on δ. `ARTExactForm.conditional_art_specified` converts the exponent by the two-sided chain rule while retaining $s =$ slack in that exponent. The code assignment, prefix-machine interpretation and uniform program costs remain separate obligations; the abstract weights alone are not a normalized posterior over explanations.
- `CoarseGraining.theoremB` / `corollaryB` — WP0193 Theorem B / Corollary B: regulatory coarse-graining is uncomputable, by reduction to Vereshchagin–Vitányi.
- `quine_floor`, `self_prediction_dichotomy`, `chaitin_blocks_minimality` — WP0192 Principle 1 / WP0162 Proposition 4: the three independent obstructions to a complete, exact, certifiable self-model (the middle one is an axiom-free diagonalization).
- `contrast_posterior_ranks_by_complexity` — WP0162 Q3 (Eq. 26): the contrast-fiber posterior ranks by joint simplicity, with no $2^{-\Delta}$ tilt.
- `transform_preserves_IK`, `shift_is_not_a_null`, `cheap_transform_not_null` — WP0202 Appendix A. If $y = T_i(x)$ is computable from x^* plus a short index, then $\mathrm{I}_K(x : T_i x) \geq K(T_i x) - K(\mathrm{code}\, i) - 3$ slack: a *cheaply-computable transform preserves algorithmic mutual information*. The corollary names an error that actually shipped into a draft — a **circular shift is not a null** for whole-object MAI, since $K(\sigma_s(x) \mid x^*) \leq K(s) + O(1) = O(\log n)$, so the shifted copy retains nearly all of its algorithmic information with the original. Using it as a surrogate subtracts the signal, and a real screening effect was consequently withdrawn as "indistinguishable from noise". `cheap_transform_not_null` states the contradiction directly: no ε below $K(\sigma_s x) - K(s) - 3$ slack can satisfy `IsNullFor`. (A time-aligned mutual information *rate* is a different object, for which a shift *is* a valid null; that object is not an AIT quantity and is not formalized here. Keeping the two apart is the whole point.) Satisfiability witness: `ShiftFrame`, in which the bound is tight, $\mathrm{I}_K(x : \sigma_s x) = K(x)$.
- `PushPull.theorem1_necessity` — WP0186 Theorem 1. Against a free two-sided disturbance ($\underline{D} < 0 < \overline{D}$), holding a band $[x_-, x_+]$ requires pull authority at the upper edge at least $\overline{D}$ and push authority at the lower edge at least $-\underline{D}$. Its contrapositive, `single_signed_cannot_regulate`, is the paper's headline necessity claim: a single-signed controller cannot hold the band. Theorem 2 (`theorem2_sufficiency`) shows the saturating edge rule attains those conditions, and `regulation_iff` assembles the two halves into the equivalence the paper states in pieces; `inward_iff` is the step that collapses the $\forall D$ condition to its two worst cases. Formalizing them showed these are *not* dynamical theorems: their proofs evaluate the field only at x_- and x_+, so their content is arithmetic on two boundary inequalities. The dynamics enter only through Nagumo's theorem, which is *assumed* (see below) and is never used to derive the sign conditions.

- `PushPull.nonstrict_fails_attraction` and its repair `margin_gives_uniform_decrease` — a defect in WP0186's Theorem 2, and the fix. The first is a *counterexample*: a plant meeting every hypothesis of the sufficiency theorem (band $[-1, 1]$, disturbance range $[-1, 1]$, unit authorities, saturating rule) whose closed-loop field vanishes at $x = 5$ under the admissible realization $D \equiv \overline{D}$, so the constant trajectory $x(t) \equiv 5$ never enters the band. Invariance survives, ultimate boundedness fails: the escape clause "strict inequality whenever $D < \overline{D}$" is vacuous because $D \equiv \overline{D}$ is itself admissible. The repair supplies a strict off-band margin $\varepsilon > 0$, giving $\dot{x} \leq -\varepsilon$ and hence entry in time $\leq (x_0 - x_+)/\varepsilon$. The asymmetry is real: the necessary condition is tight and non-strict, while sufficiency for attraction needs one ε of slack.

- `PushPull.sign_diverse_card_ge_two`, `PushPull.minimal_is_two` — WP0186's cardinality claim, with the distinction Lean forced into the open: the theorems bound *signs*, not counts. A regulating flow set must contain at least one flow of each sign, whence $|S| \geq 2$; and two flows, one of each sign, achieve it, so the bound is tight. Two pushes do not regulate however many there are.

- `PushPull.restoring_not_single_signed`, `PushPull.restoring_decomposition` — WP0186 Corollary 1. For $\kappa > 0$ the leaky-integrator field $-\kappa(x - x^\star)$ is strictly positive below $x^\star$ and strictly negative above it, so no single sign-constrained flow realizes it; and it decomposes explicitly into a non-negative push active below $x^\star$ and a non-negative pull active above. The single-term restoring law is a push–pull motif in collapsed notation, not a counterexample to one.

- `PushPull.lemma1_gap_identity`, with its existential restatement `lemma1_gap_is_S_existential` — WP0186's complexity-gap lemma, against the opaque `AITFrame` carrier. From (A2)+(A3) (`OffSplit`: $K(x_{\mathrm{OFF}}) = K(D_{\mathrm{pred}}) + K(\eta)$ within slack) and (A4) (`OnResidual`: $K(x_{\mathrm{ON}}) = K(\eta)$ within slack), the gap is $\Delta = S \pm 2\,\mathrm{slack}$ with $S := K(D_{\mathrm{pred}})$: the residual $K(\eta)$, possibly $\Theta(N)$, cancels. The regulator's dynamics play no role — the assumptions, not the mechanism, carry the lemma. `no_structure_no_gap` is the degenerate case: if the predictable component is trivial ($S = 0$) the gap vanishes up to slack, so a regulator that caps the amplitude of unpredictable disturbance claims no model content. **A retraction is recorded with these theorems.** An earlier draft chained the identity into ART to conclude that a sustained Δ *forces* model content $M(W{:}R) \gtrsim \Delta$. That inference is not licensed and the paper no longer makes it; the type-level tell is that I_K does not occur in the statement at all, and no arithmetic on `OffSplit`/`OnResidual` will introduce it. WP0186's Proposition 1 is now a selection claim, which belongs with `RegulatorSelection.lean`.

- `ModelOrPay.simple_regulator_cannot_model` — WP0203 Proposition 1, from the single AIT fact that module assumes, `MAICapped` ($\mathrm{I}_K(x{:}y) \leq K(y) + \mathrm{slack}$, a named hypothesis about a frame). If $K(R) + \mathrm{slack} < \Delta$ then $\mathrm{I}_K(W{:}R) < \Delta$ for *every* world: a regulator simpler than the gap it opens cannot carry Δ bits of model. Contrapositively (`model_implies_big`), a pair satisfying the modelling disjunct $\Delta \leq \mathrm{I}_K(W{:}R)$ has $\Delta \leq K(R) + \mathrm{slack}$ — "model it" implies "be big." The clamp then needs no special pleading: by `clamp_MAI_bounded_by_own_complexity`, $K(R) = O(1)$ alone caps its mutual algorithmic information with every world.

- `ModelOrPay.perpair_does_not_lift` and `multiplicity_cancels_penalty` — WP0203 Proposition 2, the multiplicity objection, as *witnesses* rather than assertions. For every k there is a family of 2^k terms each of weight at most 2^{-k} whose total is exactly 1; and $2^k \cdot 2^{-k} = 1$. A per-pair bound therefore places no useful bound on a sum over a fiber of matching size — the ART per-pair penalty is real and is exactly consumed by the multiplicity. These are

counterexamples to the lifting step; they establish no forward tail bound and are not evidence for one.

- `ModelOrPay.invertible_bound_insufficient` — a second *insufficiency* witness. The invertible-coupling inequality $\mathrm{I}_K(W{:}R) \geq K(R) - K(x_{\mathrm{ON}}) - O(\log)$ bounds in $K(R) - K(x_{\mathrm{ON}})$, not in $\Delta = K(x_{\mathrm{OFF}}) - K(x_{\mathrm{ON}})$: the values $K(R) = 1$, $K(x_{\mathrm{ON}}) = 0$, $K(x_{\mathrm{OFF}}) = 100$, $\mathrm{I}_K = 1$ satisfy it while leaving I_K far below the gap. The two bounds coincide exactly when $K(x_{\mathrm{OFF}}) \leq K(R)$ (`invertible_reaches_gap_iff`), which is the hypothesis nothing supplies and a clamp violates. What is machine-checked is that the invertible-coupling bound is *insufficient*; nothing here establishes the positive claim it was meant to support.

- In `ReversibleReconstruction.lean`, `initial_from_complete_records` and `null_from_complete_records` — WP0203 v20's reconstruction proposition. The first result composes recovery of the final joint state with a supplied recovery rule that inverts the joint evolution at the admitted initial state (the paper's $G(F^N(s), N) = s$ for admitted s; a global left inverse supplies that identity, and only the identity at s_0 is used). The second decodes the initial world and applies the specified null protocol. The resulting conditional complexity is bounded by the sum of the record, inverse, world-decoder, and null-decoder costs. Standard AIT bounds for applying fixed computable maps are explicit `ConditionalMapBound` hypotheses; concrete machine coding is not constructed. Joint records include regulator memory: reversibility does not imply recovery from the world-side projection alone. Since v24.2, GART expands to Good Algorithmic Regulator Theorem ("Global" in v20–v24.1) and names the existing output record balance; grounding remains its measurement extension.

- In `ActionLog.lean`, `trajectory` is the exact integer recurrence $x_{n+1} = x_n + d_n + u_n$ of WP0203's Round 1 worked case (the thermostat with an action log), and `reconstruct_null` is the decoder identity: subtracting the accumulated actions from the regulated trajectory gives the null trajectory at every step. At the frame level, `residual_from_action_log` bounds the residual by the log's description cost given the regulated record and the initial regulator plus the fixed decoder and one composition overhead; `residual_with_preload` substitutes the stored-prefix bound $K(U \mid y_R, R) \leq m + b_1$; `gap_from_disturbance` gives $\Delta \geq N - r - (c_1 + c_2)$ from an incompressible disturbance recovered from the null record and a simple regulated record; and `preload_certifies_initial_information` combines GART's world form, the residual bound and the gap bound into $\mathrm{I}_K(W{:}R) \geq N - m - (r + b + c + h)$. The finite-record section identifies the maps behind those premises and proves that they produce the records: `decode_regulated_eq_null` (the decoder on `Fin` records returns the null record), `reactive_regulated_zero` and `reactive_observes_disturbance` (the reactive rule $u_n = -(x_n + d_n)$ keeps the record at zero and is realized from the measurement), `reactive_log_eq_prefix_suffix` (the log is the negated record assembled from a stored prefix and a supplied suffix), `firstDiff_null_eq_disturbance` and `cumulate_disturbance_eq_null`, with the sum lemmas `trajectory_zero_disturbance`, `trajectory_zero_action` and `sum_fin_truncate` behind them. The bridges `hdec_from_decoder`, `hlog_from_prefix_suffix`, `hrec_from_first_difference` and `hcum_from_cumulative` derive the helpers' numerical premises from `ConditionalMapBound` and `DescriptionMapBound` for the encoded maps (the log map takes the suffix-first pair that `CondChain` bounds). The reactive case gets two-sided bounds, `reactive_residual_two_sided`, `reactive_gap_two_sided` and `reactive_initial_information_bounded`: with the frame's slack read as $O(\log N)$ they give $L = K(d \mid C, N) + O(\log N)$ and $I = O(\log N)$; the paper's $O(1)$ reading is a choice of uniform constant costs for the fixed programs these instances consume, stated beside the citation. The binary literal-description premise behind `hsuffix`, the stored-prefix fact behind `hprefix`, and one uniform decoder and assembly algorithm over the admitted horizons are paper-level obligations.

- In `GenerativeRegulation.lean`, `compensation_reversible` and `model_cancellation` check that subtracting a generated disturbance while retaining its model is invertible and cancels an exact prediction. `joint_complexity_preserved` composes a supplied inverse with standard `DescriptionMapBound` hypotheses. `model_residual` bounds the null-output reconstruction cost by reading an available model and applying its generator; `model_initial_information` inserts that bound into the existing finite GART inference. These are WP0203 v22's ideal model example: physical actuator capability and concrete universal-machine encodings remain explicit manuscript assumptions.

- In `ResidualTransfer.lean`, `class_mass_transfer` (finite family, real weights) and `class_mass_transfer_tsum` (countable family, $\mathbb{R}^{\infty}_{\geq 0}$) state that if every explanation obeys a GART-type inequality $\Delta \leq I + L + s$, then the weight of the class $\{I \leq \Delta - k\}$ is at most the weight of $\{L \geq k - s\}$; monotonicity of a nonnegative sum under set inclusion. `residual_bounded_class_mass_zero` adds a uniform residual bound and empties the low-information class. The helper `post_nonneg` records that the abstract weight `AITProb.post` is nonnegative. `conditional_tail_given_residual` states ART's withdrawn display in its original shape, conditioned on a residual bound η: the weight of $\{I \leq \Delta - k,\ L \leq \eta\}$ is at most $2^{-(k-\eta-s)}$ times the weight of $\{L \leq \eta\}$. With positive conditioning mass, their ratio is at most $2^{-(k-\eta-s)}$ and is 0 for $k > \eta + s$. The factor packages a deterministic cutoff; it supplies no separate statistical decay law. `posterior_residual_transfer` and `posterior_residual_bounded_zero` instantiate these on the abstract complexity weight `AITProb.post` (not by itself a normalized canonical-code posterior; the specified-program bridges supply that reading) for a finite family of explanations (W_i, R_i) of one observed output x with null outputs of common complexity b, taking the per-explanation inequality from `GroundedRegulation.grounded_inequality` under its named chain-rule and data-processing hypotheses: the posterior mass with $I_K(W{:}R) \leq \Delta - k$ is at most the mass with residual $\geq k - 4$ slack, and is 0 under a residual bound λ for $I_K < \Delta - \lambda - 4$ slack. For WP0203 v25, `class_mass_transfer_variable_tsum` permits both the gap and coding allowance to depend on the explanation. The strict-event consequence is `class_mass_residual_confidence_tsum`, and its uniform-residual zero case is `residual_bounded_class_mass_zero_tsum`. `gart_mass_transfer_tsum` supplies the balance from the named AIT hypotheses before summing a countable class. A normalized conditional posterior is one choice of these weights; all conditioning must use the same positive evidence denominator. An exponential information-deficit tail requires a separately justified residual tail under that posterior. The concrete clamp-family mass remains a paper-level construction. `conditional_confidence_transfer` retains the strict endpoints $I < d - \eta - h$ and $L > \eta$ under the same evidence. `conditional_variable_transfer` retains a varying allowance inside the residual event. `conditional_exponential_transfer` transfers a separately supplied exponential residual tail, evaluated at $k - h \geq 0$; it does not derive that tail from GART. All three condition on positive finite evidence mass and need their pointwise hypotheses only on its positive-weight support. The finite `posterior_residual_transfer` and `posterior_residual_bounded_zero` compare abstract `AITProb.post` weights at allowance $4s$; their names do not supply a normalized program-posterior interpretation. For WP0203 v26, `conditional_small_residual_zero` and `conditional_deficit_zero` form the conditional probability explicitly, requiring positive finite mass for the evidence and residual-budget event. GART is needed only on positive-weight members of that event. `conditional_art_tail` gives ART's exponential envelope as a consequence of the stronger cutoff. The helper `conditionalMass_le_one` verifies normalization. The guard `unadjusted_threshold_counterexample` gives a one-point law with $\Delta = n + 1$, $I = n$, and $L = 1$: GART alone does not make $I < \Delta$ rare at large gaps. It is an arithmetic guard, not a concrete universal-machine realization.

- In `IndependentRegulation.lean`, `independent_information_moment` derives $\mathbb{E}\,2^{I_K(W:R)} \le ab$ from the named classical hypotheses `CodingDominated` for both marginal laws and `PairKraft` for encoded pairs. Under normalized product weights, `independent_regulation_probability` bounds the probability of $\Delta - L - h \ge t$ by $\min\{1, ab2^{-t}\}$, allowing h to vary between pairs. `independent_regulation_threshold` needs a uniform allowance only on the event $\{\Delta \ge d, L \le \ell\}$. `independent_gart_probability` derives its pointwise premise from `grounded_inequality`, with allowance 4 slack. `independent_regulation_given_residual` conditions on a positive-mass residual-budget event and retains its probability in the denominator. This selection can remove the exponential rarity of the unconditioned joint event. These are prospective bounds under independently sampled initial descriptions, separate from ART's joint-prior posterior. Computability-to-coding domination and prefix Kraft remain named classical AIT inputs; the exponential moment and regulation consequences are proved from them. `threshold_from_local_gart` requires GART only on the positive-weight support where $L \le \eta$. `given_residual_from_local_gart` divides the resulting joint-event bound by $q_\eta = \Pr(L \le \eta) > 0$ and caps it at one. `product_event_mass_le_one` derives the finite event mass from normalized marginals; finiteness is not an independent probability assumption in that application.

- In `ARTExactForm.lean`, `exact_exponent` proves from two-sided chain rules and computability of both outputs that $K(x) - K(\langle W, R\rangle) = -(K(R \mid x) + L + H) \pm 4$ slack, with $L = K(y_\varnothing \mid x, R)$ the counterfactual residual and $H = K(W \mid x, R, y_\varnothing)$ the hidden-world term; `art_exact_form` turns it, through the coding theorem, into the two-sided bound on `AITProb.post`, `art_exact_upper` drops $H \ge 0$, and `residual_dominates_gap` shows $K(R \mid x) + L \ge \Delta + K(R \mid W) - 7$ slack, so the upper half implies the original gap form of ART. `shared_information_exponent` is the gap form made exact (the dropped term is $K(W \mid y_\varnothing)$), and `bridge_identity` equates the two exact forms using the complexity differences $K(R) - K(R \mid x)$ and $K(W \mid y_\varnothing) - K(W \mid x, R, y_\varnothing)$, within $5s$. `information_difference` and `conditional_information_difference` convert those differences into symmetric mutual information with explicit pair and context conversion costs. Thus `bridge_symmetric_information` gives $I_K(W{:}R) = \Delta - L + I_K(R{:}x) + I_K(W{:}(x, R) \mid y_\varnothing)$ within $7s + c_{\text{pair}} + c_{\text{condpair}} + c_{\text{context}}$, at most $10s$ when each conversion cost is at most s. `four_term_exponent` retains the hidden-world cost $H_0 = K(W \mid y_\varnothing)$. With the relaxed `RegulationBalance.CondMono`, `excess_nonneg` bounds the excess $\varepsilon = I_K(W{:}R) - (\Delta - L)$ below by $-7s$. `art_with_residual` has lower exponent $-K(R) - L - H_0 - 8s$ and upper exponent $I_K(W{:}R) - K(R) - \Delta + s$; `art_with_residual_specified` shifts both by the selected program's length offset $-\delta$. Their separation is $\varepsilon + H_0 + 9s$, before the distinct coding constants are included. `log2_sandwich` takes logarithms and retains the terms $-\log_2 c_2$ and $-\log_2 c_1$ at the two endpoints. This is WP0203's "Algorithmic Regulator Theorem: exact form": the posterior charges the residual bit for bit, which is why the clamp family (2^N worlds at 2^{-N} each) defeats the aggregate tail and why conditioning on the residual repairs it.

- In `HorizonConditioning.lean`, `conditioning_on_horizon` removes the horizon from the conditioning of shared information: with `CondMono` for the two marginals and `Subadd` for the pair against the horizon object N, $\mathrm{I}_K(W{:}R) \ge \mathrm{I}_K(W{:}R \mid N) - K(N) - 3$ slack (the two ignore-input bounds and the subadditivity each carry one slack; together they are the paper's fixed horizon-conditioning cost c_0). `horizon_bound_every` composes this with the per-horizon initial-information bound $\mathrm{I}_K(W{:}R \mid N_i) \ge \Delta_i - \lambda_i - h_i$ (WP0203's initial-information corollary with the horizon explicit) to give $\mathrm{I}_K(W{:}R) \ge \Delta_i - \lambda_i - h_i - K(N_i) - 3$ slack at every admitted horizon, and `horizon_bound_sup` states the supremum form over any nonempty finite family of horizons. This is WP0203 v32.10's remark on reading the bound across horizons, which the paper had left at paper level; the background C_0 is the frame and the horizon is an object.

- In `InterventionKnowledge.lean`, `key_bound` proves that a descriptor m recovered by fixed maps

from both W and R forces $\mathrm{I}_K(W{:}R) \geq K(m) - 3$ slack (data processing in each argument, then `SelfInformation`); `intervention_and_gap` places this beside `grounded_inequality` as a maximum, not a sum. `readout_saving_upper` bounds the description saving $K(Z) - K(Z \mid R)$ of any world record Z computable from W by $\mathrm{I}_K(W{:}R) + 2$ slack; `descriptive_saving` brackets the saving on the null output between $\Delta - L - 2$ slack and that bound; and `shared_content_split` is the two-sided chain-rule decomposition $\mathrm{I}_K(W{:}R) = \mathrm{I}_K(y_\varnothing{:}R) + \mathrm{I}_K(W{:}R \mid y_\varnothing) \pm 3$ slack from three named chain facts. These are the four displays of WP0203 v34.3's Discussion subsection on intervention knowledge and descriptive power, previously paper-level.

- `RegulationBalance.balance`, `balance_readout`, `deficit`, `balance_rate_finite`, `gap_forces_exhaust` — WP0203 v2, the regulation balance: the positive replacement for the withdrawn concentration corollary. With Q bundling the regulator's action transcript, its retained private state and the *world exhaust*, and under an information-closure hypothesis $K(y \mid \langle Q, \langle x, R\rangle\rangle) \leq$ slack that makes Q an admissible completion,

$$\Delta \;\leq\; \mathrm{I}_K(W{:}R) + \mathrm{cIK}(y{:}Q \mid \langle x, R\rangle) + O(\text{slack}).$$

  Equivalently (`deficit`) the episode flow is bounded *below* by $\Delta - \mathrm{I}_K(W{:}R)$: the distinctions removed from the scored readout do not disappear. `gap_forces_exhaust` is the trichotomy — no model content, no world-relevant action transcript and no retained state force the whole gap into the exhaust term, which is the regime the clamp family of `ModelOrPay` occupies. In WP0203 v23, the paper fixes the record-selection rule in advance and requires records from the actual regulated episode. The formal balance consumes their recoverability premise; physical completeness and record selection remain manuscript assumptions. The limiting rate argument is analysis on top of `balance_rate_finite` and is not separately formalized. `residual_in_completion` is the supporting step converting "there is a residual" into "the residual sits in Q."

- `RegulationBalance.residual_with_missing_information` splits $K(y \mid C)$ into $\mathrm{cIK}(y{:}Q \mid C)$ and $K(y \mid Q, C)$, within the supplied two-sided conditional chain-rule tolerance. This is WP0203 v23's incomplete-reconstruction identity, with no reversibility premise.

- `balance_with_missing_information` retains the missing conditional complexity in the regulation bound when the selected records are insufficient. Only the upper conditional chain rule is needed for this inequality; it assumes neither reconstruction nor reversibility. Neither this bound nor the preceding identity equates missing information with physical erasure.

- `RegulationBalance.localized_closure_needs_world_record` and `no_localized_exhaust_of_world_record_insufficient` — WP0203 v4's *localization* clause, which is what stops the balance closing by fiat. If the exhaust were an arbitrary string the choice $\Xi := y$ would satisfy recovery trivially; v4 requires Ξ to be a fixed projection of the realized world-side record S. At frame level that shadow is `Localized` ($K(\Xi \mid S) \leq$ slack), and what is machine-checked is the anti-vacuity content: a localized exhaust cannot manufacture recovery the world record does not already support, so if S with the visible episode fails to determine y, no localized Ξ closes the accounting. The physical-state structure — what a world-side record *is*, and that the projection is fixed independently of y — is not expressible in an `AITFrame` and is not formalized.

- `RegulationBalance.cyclic_form` — WP0203 v4's cyclic corollary. A regulator returning its private state to a standard state drops the memory channel, leaving model content, action transcript, and world exhaust: *model it, transmit it, or leave it in the world.*

- `PersistenceFlow.algorithmic_persistence_balance` and its suite give the **Algorithmic Persistence Balance (APB)**. With completion $Q = (Z^+, \mathcal{W}^+, \sigma, a, \Xi)$ and initial context $\mathsf{Ctx} = (C, Z_t, \mathcal{W}_t, \sigma_t)$ (frame, self-code, model, and retained memory), the explicit-error form `algorithmic_persistence_balance_with_error` assumes $K(\iota \mid Q, \mathsf{Ctx}) \le \delta$ (`ReconstructionBound`), a conditional chain rule, and the ordered split `FlowSplit5`. It proves

$$K(\iota \mid \mathsf{Ctx}) \le J_Z + J_M + J_\sigma + J_a + J_\Xi + \delta + 2s,$$

where s bounds each named frame slack. The reconstruction allowance δ is independent of s. The original `algorithmic_persistence_balance` is the $\delta = s$ specialization and retains its $3s$ remainder. The explicit-error corollaries `bounded_persistence_forces_flow_with_error` and `flow_pigeonhole_with_error` cap the internal channels using `CIKCeiling`. With d_Z, d_M, d_σ their conditional description costs,

$$J_a + J_\Xi \ge \nu - d_Z - d_M - d_\sigma - \delta - 5s, \qquad 2\max\{J_a, J_\Xi\} \ge \nu - d_Z - d_M - d_\sigma - \delta - 5s.$$

Thus a positive residual forces information into the two external channels, conditional on the preceding records. The older `bounded_persistence_forces_flow` and `flow_pigeonhole` retain their $6s$ remainder under `InfoClosed`. No temporal conservation or physical localization premise is proved by these accounting statements.

`self_code_overload` gives $K(Z_1 \mid Z_0, C) \ge K(\iota \mid Z_0, C) - \delta - s$ by conditional subadditivity. `bounded_self_code_update_capacity` uses a conditional update budget $K(Z_1 \mid Z_0, C) \le K_0$ to bound the recoverable novelty by $K_0 + \delta + s$; `bounded_self_code_capacity` instead assumes $K(Z_1) \le K_0$ and `CondMono`. Finally, `persistence_does_not_imply_gap` gives a finite frame witness (`noGapFrame`) with $\mathrm{NMAI}(z, z) = 1$, $\mathrm{NMAI}(z, 0^n) = 0$, and raw gap $\Delta = K(0^n) - K(z) < 0$. This is a witness in the abstract frame interface; it does not construct an optimal universal machine realizing those exact assigned complexities.

- `CompletionLedger.grouped_completion_balance` and its suite state the reconstruction-and-capacity account of WP0216's APB and WP0203's residual identification once, with neutral names, for the WP0218 companion revision (October 2026). Grouped form: with target D, initial context B, internal record S, and complementary record E, conditional subadditivity through S (`CondSubadd`), the conditional chain rule for $\langle D, E\rangle$ given $\langle S, B\rangle$ (`CondChain`), and reconstruction $K(D \mid E, S, B) \le \delta$ (`ReconstructionBound`) give

$$K(D \mid B) \le K(S \mid B) + \mathrm{I}_K(D{:}E \mid S, B) + \delta + 2s;$$

an internal budget $K(S \mid B) \le b$ then forces $\mathrm{I}_K(D{:}E \mid S, B) \ge K(D \mid B) - b - \delta - 2s$ (`grouped_bounded_internal_forces_external`), and with no complement $K(D \mid B) \le b + \delta + s$ (`grouped_internal_capacity`; the self-code capacity theorem is the case $S = Z_1$, $B = \langle Z_0, C\rangle$). Ordered form: for records supplied in a declared order, `ctxAfter`, `orderedInfo`, and `orderedCost` give the nested context, the ordered conditional-information sum, and the ordered description cost; `OrderedSplit` is the named split hypothesis generalizing `FlowSplit5`, and `OrderedCeilings` caps each internal channel. `ordered_completion_balance` gives $K(D \mid B) \le \sum_i J_i + \delta + 2s$; with k internal records, `ordered_bounded_internal_forces_external` gives $\sum_{\mathrm{ext}} J \ge K(D \mid B) - \sum_{\mathrm{int}} d - \delta - (2 + k)s$, and `ordered_two_external_pigeonhole` the max form for two external records (`orderedInfo_append` and `orderedInfo_le_orderedCost` are the list lemmas). The allowances stay distinct: δ is the reconstruction allowance, s bounds each named estimate including the whole ordered split, and $(2 + k)s$ counts hypothesis uses. The five-record APB is the instance $[Z^+, \mathcal{W}^+, \sigma, a, \Xi]$ with $k = 3$: `flowSplit5_iff_orderedSplit` identifies the split hypotheses, `orderedCost_internal_three` the internal cost $d_Z + d_M + d_\sigma$, and `apb_flow_`

`from_ordered` re-derives the conclusion of `bounded_persistence_forces_flow_with_error` with $(2+3)s = 5s$. No temporal conservation or reversibility premise appears; the account applies to irreversible realizations. The whole-split hypothesis is itself derived (October 5, WP0218 review C04): `orderedInfo_telescope` telescopes the per-step easy chain rule (`ChainAlong`) to $\sum_i J_i \geq K(D|B) - K(D|B_n) - n\,s$; with the hard chain rule on the joint record, allowance $h_{\log}$ (`JointChainLower`), and the conversion between the nested and tuple contexts, allowance h_{conv} (`NestedConversion`), `orderedSplit_telescoped` gives split error $h_{\log} + h_{\text{conv}} + n\,s$, and `ordered_completion_balance_telescoped` gives the balance with remainder $\delta + h_{\log} + h_{\text{conv}} + (n+1)s$. For the classical prefix machine $h_{\log} = O(\log(N+2))$ with $N \geq n + K(D, Q|B)$, and $h_{\text{conv}} = O(1)$ when n is supplied, $O(\log(n+2))$ otherwise.

- `StateGraph.unique_iff_permGraph` and its suite check the support-level form of Susskind's conservation of information used in WP0218's Appendix E. For a transition-support relation R on any state space (the empty one included), every state has exactly one allowed successor and exactly one allowed predecessor (`UniqueSucc`, `UniquePred`) if and only if R is the graph of a permutation (`IsPermGraph`; `permGraph_of_unique`, `unique_of_permGraph`). The exact-k-step relation `Steps` is then the k-th power of that permutation (`steps_iff_pow`), so every exact-time future and past layer is a singleton (`steps_unique_succ`, `steps_unique_pred`), and the permutation's orbit label from `OrbitLabel` is conserved along every allowed transition (`orbit_label_of_unique`). Pure relation and permutation algebra, no AIT frame. The weaker bounded-advice form that WP0218's reversible invariance lemma consumes is deliberately not identified with the singleton condition.

- **Compressive models in the completion account.** `ModelAccounting.lossless_recode_complexity` bounds the conditional-complexity change between mutually reconstructible records by $\varepsilon + s$. `model_parameter_residual_bound` charges an acquired model, its finite parameters, and its residual in order; three named subadditivity estimates give their summed conditional cost plus $\delta + 3s$. With $\kappa = K(\theta \mid B)$, `innovation_lower_bound` gives $K(D \mid B) \geq \kappa + n - c - \varepsilon - 2s$ when the record recovers the parameter θ and an innovation string having conditional complexity at least $n - c$. If $K(S \mid B) \leq \kappa + b$, `innovations_force_complement` gives $I_K(D : E \mid S, B) \geq n - c - b - \varepsilon - \delta - 4s$ under grouped reconstruction; `innovations_limit_internal_retention` gives $n \leq b + \delta + c + \varepsilon + 3s$ when internal records alone reconstruct the target. All coding and incompressibility statements enter as named or explicit hypotheses. Here c is the innovation deficiency, ε the pair-recovery allowance, δ the target-reconstruction allowance, and b the internal cost beyond κ. The finite list results `block_residual_recovery`, `block_parameter_recovery` (positive horizon), `block_record_recovery`, and `block_stream_length` check the construction with a q-bit repeated block; they do not establish its Kolmogorov complexities or a model-discovery guarantee.

- `GroundedRegulation.grounded_readout`, `grounded_inequality`, `repaired_corollary`, `repaired_good_regulator` — WP0203's grounded exposition, the repaired static-MAI corollary. A designated reguland trajectory z (ON) and z_0 (OFF) replaces the scored readouts; `Grounded` ties $x \leftrightarrow z$ and $y \leftrightarrow z_0$ up to tolerance ε in algorithmic information distance (`gap_transfer`: the readout gap and the reguland gap $\Delta_Z = Kz_0 - Kz$ then agree up to $2\varepsilon + O(\text{slack})$, which blocks the thermometer hack). With the counterfactual residual $L_Z = K(z_0 \mid \langle z, R\rangle)$ (`residual`),

$$\Delta_Z \;\leq\; \mathrm{I}_K(z_0{:}R) + L_Z + O(\text{slack}) \;\leq\; \mathrm{I}_K(W{:}R) + L_Z + O(\text{slack}),$$

so under grounding $\mathrm{I}_K(W{:}R) \geq \Delta - L_Z - 2\varepsilon - O(\text{slack})$: the original Good-Regulator reading is recovered exactly when the readout is grounded and the intervention is counterfactually

non-collapsing (L_Z small). The proof spine is `RegulationBalance.balance_readout` stopped at the residual instead of closed by Q.

- `GroundedRegulation.simple_regulator_forces_residual` and `simple_regulator_forces_residual_readout` — the ceiling $\mathrm{I}_K(W{:}R) \le KR + \text{slack}$ (`IKCeiling`, standard, named hypothesis) turns the grounded inequality around: $L_Z \ge \Delta_Z - KR - O(\text{slack})$. The readout form applies the ceiling to $\mathrm{I}_K(z_0{:}R)$ in `grounded_readout` and needs no null-output computability; it is WP0203's Eq. (forcesL), with allowance 4 slack (the world form carries 5 slack). A fixed simple regulator opening an extensive grounded gap is *forced* into an extensive counterfactual residual — the blind clamp/AC regime classified, and the structural reason no horizon-uniform $\mathrm{I}_K(W{:}R) \succ \Delta_N$ can hold for fixed finite R: static model content is reusable, cumulative regulation benefit is not.

- `GroundedRegulation.closure_by_construction` — WP0203 v10's Remark (automatic closure). In the deterministic Turing-pair idealization the persistent world description is instantiated in the realized world-side record S and the null readout is computable from it, so $K(y \mid S, C) \le$ 4 slack for any visible episode C: an information-closing record exists *by construction*, via the named conditional composition fact `CondCompose` ($K(y|S) \le K(W|S) + K(y|W) + \text{slack}$) and conditioning monotonicity. Closure is therefore automatic in abstract ART and becomes a substantive hypothesis only for physically bounded realizations; localization and minimality retain the substantive bookkeeping role.

- `GroundedRegulation.residual_is_completion` and `grounded_balance` — the bridge back to the balance. Under the reguland-level closure with completion $Q = \langle O_R, \sigma_R, \Xi\rangle$, $|L_Z - \mathrm{cIK}(z_0{:}Q \mid \langle z, R\rangle)| \le 2$ slack (the second direction needs `PairMono`, $K(B \mid C) \le K(\langle A, B\rangle \mid C) + \text{slack}$): the residual is not a new sink but exactly the completion term, so the repaired corollary (when L_Z is small) and the WP0203 balance (where L_Z goes when it is not) are two views of the same accounting. `closure_transfers` transports the reconstruction condition to a selected variable (WP0203 Eq. zclosure-early): with the two grounding translations $K(y_R \mid \zeta_R) \le \varepsilon$ and $K(\zeta_\varnothing \mid y_\varnothing) \le \varepsilon$, the reconstruction bound `InfoClosed`, two compositions and three context conversions written out as instances, $K(\zeta_\varnothing \mid \zeta_R, R, Q) \le 2\varepsilon + 6$ slack; the conclusion is not an `InfoClosed` at tolerance slack, and a nonlogarithmic ε is not absorbed. `grounded_balance` is `RegulationBalance.balance` instantiated at $x := z$, $y := z_0$, recorded to show the constructions compose. The asymptotic $(1 - o(1))\Delta$ form and the $\Theta(N)$ readings are analysis on top of these finite inequalities and are deliberately not formalized.

- `GroundedRegulation.exact_counterfactual_balance`, `gart_reguland_from_exact_balance`, `grounded_inequality_from_exact_balance`, `exact_counterfactual_balance_closed` — WP0203 v16's equality-level ledger, the conservation seam. With all quantities conditioned on the declared common frame C and $L_{\mathrm{cf}} = K(\zeta_\varnothing \mid \zeta_R, R, C)$ (`residualC`),

$$\Delta_N^\zeta = \mathrm{I}_K(\zeta_\varnothing{:}R \mid C) + L_{\mathrm{cf}} - \mathrm{I}_K(\zeta_R{:}R \mid C) - K(\zeta_R \mid \zeta_\varnothing, R, C) + O(\text{slack}),$$

  a two-sided finite-slack identity assembled from conditional symmetry of information (`CondSI`) and the conditional chain-rule exchange (`CondExchange`) — pure finite-string bookkeeping, deliberately carrying *no* reversibility premise (WP0218 supplies the physical complete-ledger reading in the manuscripts, not an algebraic hypothesis here). The GART inequality is then re-derived as its *projection*: dropping the two regulated-branch correction terms ($K(\zeta_R \mid \zeta_\varnothing, R, C) \ge 0$ and $\mathrm{I}_K(\zeta_R{:}R \mid C) \ge -$slack, `MutualNonnegC`) and composing with conditional data processing (`DataProcessingC`) gives $\Delta_N^\zeta \le \mathrm{I}_K(W{:}R \mid C) + L_{\mathrm{cf}} + O(\text{slack})$, so the formal dependency graph matches the manuscript narrative equality → projection → inequality. Substituting `residual_is_completion` at conditioning base $\langle R, C\rangle$ closes the ledger: under

reguland-level closure the residual term *is* $\mathrm{cIK}(\zeta_\varnothing{:}Q \mid \zeta_R, R, C)$ — the residual is not a sink but the matched-null information localized in the complementary realized records.

- `GroundedRegulation.self_regulation_forces_self_model` and `self_regulation_forces_self_model_grounded` — WP0203 v13's self-regulation corollary. The query is role-relative: the focal pattern decomposes as $P = R \cup H$ $(R \cap H = \varnothing)$ and the reguland projection ζ^P tracks the self-side region H, so the implicit task-relevant self-model content $\mathsf{SM}^{\mathrm{imp}}_{P,N}(R) := \mathrm{I}_K(\zeta^P_\varnothing{:}R \mid C)$ is the `IK` $(z_0{:}R)$ of the grounded inequality read at $z := \zeta^P_R$, $z_0 := \zeta^P_\varnothing$. The reguland form rearranges `grounded_readout` to $\mathsf{SM}^{\mathrm{imp}} \geq \Delta_Z - L_Z - O(\mathrm{slack})$; the grounded form composes with `gap_transfer` to give $\mathsf{SM}^{\mathrm{imp}} \geq \Delta - L_Z - 2\varepsilon - O(\mathrm{slack})$ against the scored gap. The decomposition of P and the word "self" are manuscript semantics layered over the same string inequality — no new ontology, no new axiom, and no claim of explicit symbolic self-representation or self-targeting.

- `Localization.localization_sum_exact`, `localization_delta_exact`, `marginal_delta_exact`, `localization_conserved_exact` — WP0218's Level-1 algebra. The localization coordinates $L_A = K(A,W) - KW$, I_K, $L_W = K(A,W) - KA$ are an invertible reparameterization of the standard pair complexity profile with $L_A + I_K + L_W = K(A,W)$ exactly; between endpoints the coordinate sum changes exactly by the joint-budget change, equivalently $\Delta K_A + \Delta K_W - \Delta I_K = \Delta K(A,W)$ (the Ebtekar–Hutter two-system decomposition), and a fixed joint budget conserves the sum exactly. Abstract `Int` identities — notation checks, deliberately claiming no new invariant; `localization_mutual_eq_IK` ties the abstract mutual coordinate to the frame's `IK` definitionally.

- `Localization.reversible_complexity_invariance` and `localization_balance_reversible` — the reversible wrapper and WP0218's Algorithmic Localization Balance (reversible specialization). Mutual ε-bit reconstruction plus plain subadditivity both ways gives $|Ky - Kx| \leq \varepsilon + \mathrm{slack}$; applied to a fixed partition's paired endpoints, the localization sum moves by at most $\varepsilon + \mathrm{slack}$ while the marginals redistribute freely.

- `Localization.recoverable_description_overlap`, `recurrent_recoverability_implies_mai_bound`, `recurrent_recoverability_persistence_bound` — WP0218's overlap lemma and its temporal instantiation. From `MutualChain` and the new named standard fact `CondDataProcessing` $(\mathrm{I}_K(D{:}Y) \leq \mathrm{I}_K(X{:}Y) + K(D|X) + \mathrm{slack})$, $\mathrm{I}_K(X{:}Y) \geq KD - K(D|X) - K(D|Y) - 2\,\mathrm{slack}$: a description cheaply recoverable from both sides is paid for by their MAI. Instantiated at pattern descriptions at two times with ε-recoverability of a named D_P, this lower-bounds cross-time MAI and (dividing by the larger endpoint complexity) the WP0216 persistence score. One direction only: no converse extraction of generic MAI is stated, and recurrent recoverability is a sufficient mechanism for persistence, not its definition. The plain-conditioned sibling of `IS.common_semantics_bound` (which conditions on shortest programs); the two consume different named hypotheses and are kept separate.

- `Localization.recoverable_description_overlap_steps` derives the overlap lemma's data-processing step instead of assuming it (WP0218 review, October 2026). From starred symmetry of information in both directions (`SymStarUpper`, `SymStarLower`), recovery of X from X^* (`StarRecovers`), subadditivity through D (`SubaddThrough`), and the plain-to-starred conversion with an explicit allowance h (`StarConversion`), `cond_data_processing_from_steps` gives $\mathrm{I}_K(D{:}Y) \leq \mathrm{I}_K(X{:}Y) + K(D|X) + h + 4s$, and the overlap and recurrent bounds follow with remainder $h + 5s$ (`recurrent_recoverability_steps`). The logarithmic allowance h is the manuscript's $O(\log(N+2))$, kept separate from the constant slack s. The original named hypothesis is not recovered with a single frame slack; papers should cite the explicit-error

forms.

- `TokenTransport` — WP0218's canonical token transport calculus, finite and Mathlib-free. `cnotAW_involutive`, `cnotWA_involutive`, `swap_involutive`, and `gateFor_involutive` check the reversible gate vocabulary; `canonical_mode_reachable` shows one gate converts any canonical mode $\mathsf{A}(x) = (x, 0)$, $\mathsf{AB}(x) = (x, x)$, $\mathsf{W}(x) = (0, x)$ into any other; `transport_plan_exists` builds the northwest-corner 3×3 plan between any equal-total profiles; `transport_plan_realizable` realizes any plan entrywise by disjoint token-pair gates (an involution, `applyGates_involutive`, so the circuit is reversible); `diag_le_min`, `exists_min_moved_plan`, and `minimum_moved_mass` give the minimum moved token mass $N - \sum_i \min(p_i, p'_i) = \frac{1}{2}\|p - p'\|_1$, the ℓ_1 identity stated in doubled `Nat` form. Scope guard: canonical independent-token sector only — nothing says an arbitrary pair with a given profile factors into such tokens.

- `IS.common_semantics_bound` — WP0215's common-semantics lower bound. If a finite semantic object F is recoverable from p at cost c_1 and from q at cost c_2, both conditioned on the shortest programs p^*, q^*, then $\mathrm{I}_K(p{:}q) \geq K(F) - c_1 - c_2 - 4$ slack. Two descriptions of the same finite semantics cannot be algorithmically independent. The paper's $O(\log n)$ is the explicit `slack` term and the coefficient 4 counts the inequalities the chain actually pays: self-information (`SelfInfo`), data processing twice (`DataProcessingIK`), and one swap (`PairSymmetric`). `IS.common_semantics_bound_raw` is the same bound with the recovery costs left as the frame's own conditional complexities; instantiating F at an intervention-response table $T_D(f)$ gives the observer-relative reading. The paper's D-conditioned form $\mathrm{I}_K(p{:}q \mid D) \geq K(T_D(f) \mid D) - O(\log n)$ is *not* proved here: it needs conditional analogues of the two AIT hypotheses, which are not stated. **Two guards bound the reading.** `IS.common_semantics_does_not_give_structure` exhibits a frame where the bound is attained exactly while $K(p \mid q^*) = 97$ of p's 100 bits remain unrecoverable from q: shared semantics is not shared organization, and near-minimality — a separate hypothesis, not assumed here — would force shared *information*, never shared causal form. `IS.equal_complexity_is_not_shared_information` exhibits two descriptions of equal complexity with $\mathrm{I}_K = 0$, so equal description length is not a weak form of the hypothesis.

- `IS.boundary_eq_of_iso` — WP0215's equivalence hierarchy, the direction that holds: an internal isomorphism restricted to reachable states implies equality of the boundary map, here the full output trace on every finite input word. The converse fails, and not by dead code: `IS.parity4_boundary_eq_parity2` and `IS.parity4_not_internal_iso` give parity computed by a counter modulo four and by a counter modulo two — same boundary, every state of the four-state machine reachable (`IS.parity4_all_reachable`), no isomorphism, because four reachable states do not inject into two. Supporting: `IS.trace_eq_of_iso`, `IS.parity_trace_eq`, `IS.red_out`, `IS.red_step`, `IS.run_append`, `IS.reach_step`, `IS.reach_start`.

- `IS.finite_unfolding` — the correction WP0215 records to an earlier KT reply to the unfolding argument. For every deterministic recurrent machine and every finite horizon T there is an acyclic depth-T layered evaluator computing the same map; the witness is constructed (`IS.unfold`, `IS.unfold_valueAt`). The load-bearing modelling choice is that the state carries everything that changes, weights included, so adaptation is no obstacle: an adapting machine is a machine with a larger state. Acyclicity is typed rather than asserted, each layer producing only the next layer's carrier. `IS.unfolding_layers_eq_state` records the construction's shape definitionally and stops there — this development has no cost model, so no size or depth growth is stated.

- `IS.depth_bounded_ignores_late_input` — the honest negative half. A depth-T layered evalu-

ator is a function of the first T inputs alone, whatever it computes; `IS.evalHorizon_ignores_late_input` is the same for the recurrent machine at a fixed horizon. Hence `IS.no_fixed_unfolding_covers_all_horizons`: for an echo machine no fixed depth and no evaluator of that depth reproduces the map at every horizon, since a candidate of depth T is already wrong at $T+1$. The uniform statement — one acyclic machine, all horizons — is *not written here and cannot be*: the type `Layered X Y T` is indexed by T, so a single term of it has one fixed depth. What is proved is what the paper's argument needs, that the unfolding must be re-chosen as the horizon grows.

- `OrbitLabel.genEnergy_conserved` — WP0162 App. C: generalized energy as a conserved orbit label (axiom-free).
- `boundary_sufficient` — WP0162 App. E: the thin boundary as a compressed sufficient statistic.
- `low_complexity_shrinkage` — ART Theorem A4; `CoarseGraining.theoremA`, `CoarseGraining.existence` — WP0193 Theorem A and Proposition 1.
- `Noether.trajLabel_conserved`, `Noether.conserved_comp_symm` — the abstract core of generalized Noether (Structured Dynamics paper): the trajectory label is conserved by the flow, and symmetries commuting with the flow act on conserved quantities. (Continuous-time analogue of the orbit-label theorem.)
- `Homogeneous.homogeneousSpace` — a transitive symmetry action makes the state space a homogeneous space, $X \simeq G/H_x$ (Theorem A4's algebraic core; the manifold/dimension layer remains future work).
- `Decoder.achievable_computable_of_inverse` / `Decoder.no_universal_decoder` — WP0058 Proposition 1(i)–(ii): a *total* computable write-back encoder that recognizes its own domain would decide membership in the achievable set of the developmental map; hence none exists (`exists_map_without_decoder`), and every realized write-back channel is a partial map whose validity domain must be supplied or learned rather than recognized by the encoder itself. Notable as the one KT corollary here that assumes *no* AIT fact at all — the engine (`achievable_computable_of_inverse`) is unconditional, and only the non-vacuity of the undecidability hypothesis enters as a named `Prop` (`ExistsUndecidableAchievableSet`). Satisfiability witness: `inverts_id`.
- `Decoder.partial_inverse_exists` / `Decoder.achievable_rePred` — the two positive halves of the same proposition, which locate exactly what the impossibility costs. Dovetailing search gives a *partial* inverse that returns a genuine preimage wherever it is defined, so inversion as such is available; what is lost is the ability to *refuse*, since the search diverges off the achievable set. Correspondingly the achievable set of a computable developmental map is recursively enumerable, which machine-checks the informal half of WP0058's proof ("enumerate programs, run D, collect outputs") and leaves only undecidability assumed.
- `WriteBack.channel_bound` — WP0058's channel-bound lemma as a single named result covering both links, $\mathrm{I}_K(a : H' \mid w) \le \mathrm{I}_K(a : z \mid w) \le K(z \mid w) + O(\log)$. The right link is `condIK_le_condRight`; the left is data processing, assumed as `DataProcessingCond`. Cite this for the whole chain and `condIK_le_condRight` for the second link alone.
- `WriteBack.darwinian_of_dataProcessing` — a Darwinian regime is exactly the general data-processing hypothesis instantiated at the selection signal, so `Darwinian` assumes nothing beyond `DataProcessingCond` plus the caller's Markov premise that σ is the only route from a

into H'.

- `WriteBack.bandwidth_le_cond`, the channel bound, and the regime corollaries — WP0058 Proposition 3. Writing λ_B for the write-back bandwidth (defined below on the write-back program, *not* as $\mathrm{I}_K(a : H' \mid H)$), the master bound $\lambda_B \le K(H' \mid H) + O(\log)$ holds but is weak in the Darwinian regime, where undirected mutation makes $K(H' \mid H)$ large without transmitting anything acquired. The sharp statement is data processing (`condIK_le_condRight`): since a reaches H' only through a channel variable z, $\lambda_B \le \mathrm{I}_K(a : z \mid H) \le K(z \mid H) + O(\log)$. Crucially, the write-back bandwidth is *not* $\mathrm{I}_K(a : H' \mid H)$: that quantity is nonzero even in a strictly Darwinian lineage, since acquired structure influences *who reproduces*. Mutual information cannot isolate direct write-back — directness is causal, not correlational — so WP0058 conditions on the selection signal, $\lambda_B := \mathrm{I}_K(w : H' \mid H, \sigma)$ on the tagged write-back program $w = C(a)$, which attaches the quantity to the channel rather than inferring it from residual dependence, and blocks $a \to \sigma \to H'$ (`writeBack`). No additive split $\lambda_{\mathrm{sel}} = \mathrm{I}_K(a : H' \mid H) - \lambda_B$ is defined: conditional mutual information is not additively decomposable. Then: in the Darwinian regime there is no write-back program, so $\lambda_B \le O(\log)$ (`no_program_no_write_back`) — the premise is the *absence of the write-back program*, architectural and posited, since no information measure certifies a missing causal path — while the *total* information crossing is bounded by the selection signal, $\mathrm{I}_K(a : H' \mid H) \le K(\sigma) + O(\log)$ (`darwinian_bandwidth_le_selection`, zeroth-order): that bound is the substantive half. In the Lamarckian regime $\lambda_B \le K(w \mid H, \sigma) + O(\log)$ (`lamarckian_bandwidth_le_decoder_image`, first-order) — immediate, since λ_B is *defined* on w. `toyWB_writeback_zero_but_total_positive` machine-checks that the two quantities genuinely differ: a witness with $\lambda_B = 0$ and total $= 3$. Also `decoder_charged` ($K(C) \le K(H) + O(\log)$, Prop. 2's background-free form) and `trivial_decoder_transmits_nothing` (Corollary 1: a trivial write-back program transmits nothing however much was acquired). Witness `ToyWB` satisfies both named AIT hypotheses *and attains* the Darwinian bound (`toyWB_selection_bound_tight`) — the corollaries are tight, not merely non-vacuous.

- `WriteBack.decoder_charged_rel` — WP0058 Proposition 2 proper, the background-relative form. `decoder_charged` charges the *whole* write-back apparatus to the heritable program, but part of it is supplied by a background B: the cell, the parent, the niche, a teacher, an external training pipeline. Under conditional subadditivity relative to a background (`SubadditivityCondRel`, a named `Prop`) an encoder reconstructible from H and B jointly (`RecoverableFromRel`) costs the lineage only $K(C \mid B) \le K(H \mid B) + O(\log)$. The hypothesis is strictly weaker than `RecoverableFrom` (`toyWB_background_strictly_weaker`) and the bound is attained away from zero (`toyWB_decoder_charged_rel_tight`); `ToyWB` satisfies it (`toyWB_subadditivityCondRel`). What is *not* shown, here or anywhere in the module: that write-back cannot produce novelty, or that the encoder's representational axes were found by prior selection.

- `WriteBack.retention_le_program` — the retention curve $M(n) = \mathrm{I}_K(w : H_{g+n} \mid H_g)$ (`retention`) is bounded at every lag by $K(w \mid H_g) + O(\log)$: what survives never exceeds what was sent. A corollary of `condIK_le_condRight`.

- `Retention.bangbang_optimal` — the optimal inherited-memory kernel is a *threshold rule*. With payoff $J = \sum_{n<N} q(n)\, v(n)$ linear in a box-constrained kernel $q : \mathbb{N} \to [0, 1]$ (`J`, `Admissible`) and $v(n)$ the net value of still acting on n-generation-old information, every admissible kernel is dominated termwise by $q^*(n) = \mathbb{1}[v(n) > 0]$ (`qStar`, with `qStar_of_pos`, `qStar_of_nonpos`, `qStar_admissible`). If v is antitone the retained set is an initial segment (`optimal_kernel_antitone_is_initial_segment`), so the optimum is "retain for L generations, then drop", and

L is the *first crossing*: $q^*(n) = 1 \iff n < L$ (`qStar_eq_one_iff_lt_first_crossing`, `qStar_eq_zero_of_first_crossing_le`) for L satisfying `FirstCrossing`, which exists whenever v is somewhere nonpositive (`exists_firstCrossing`).

- `Retention.persistence_is_first_crossing` — the punchline. Instantiating $v(n) = R(n) - \kappa$ (`netValue`, antitone by `netValue_antitone`) with R the predictive information the environmental feature still carries at lag n and $\kappa > 0$ the per-generation cost of carrying it plus the mismatch loss, the optimum retains exactly the lags where $\kappa < R(n)$. Optimal persistence is therefore the first lag at which R crosses the cost threshold — WP0058's $\tau_{\rho,\theta}$ with $\theta = \kappa$. The matching relation is the *solution* of a linear retention problem, not a modelling choice, and the threshold is set by the cost.

- `Retention.exponential_crossing` — for $R(n) = R_0 e^{-n/\tau}$ with $R_0, \tau > 0$ and $\kappa > 0$, $\kappa < R(n) \iff n < \tau \log(R_0/\kappa)$, so the optimal kernel is the indicator of that interval (`qStar_exponential`) and the crossing lag is *proportional* to the environmental correlation time at fixed cost ratio (`exponential_persistence_proportional_to_tau`). Non-vacuity: `vTest_antitone`, `vTest_firstCrossing` and `toyRetention_qStar_beats_full` exhibit an antitone net value whose optimum is a nontrivial initial segment and strictly beats full retention ($3 > 2$). What is *not* proved: that any real channel sits at this optimum, that the payoff is linear in q, that costs are constant in n, or that decay is exponential. The empirical matching claim remains Hypothesis 1.

- `persistence_conservation` / `conservation_tradeoff` — WP0162 Proposition 2: the ledger $K(O_W) = \mathrm{I}_K(O_W{:}R) + K(O_W \mid R^*)$ (from symmetry of information), and at fixed $K(O_W)$, maximizing I_K = minimizing the conditional residual.

- `meta_persistence` / `meta_persistence_limit` — WP0162 §6 (meta-persistence): a collective submodel of stable complexity $k > 0$ with a bounded transient $K(S_t^C \mid (S_{t+\tau}^C)^*) \le L$ (delivered by closure + a Foster–Lyapunov contraction of the constituent flow) has collective persistence $\mathrm{Pers}_C \ge 1 - (L + \mathrm{slack})/k$, hence $\to 1$ as $k \to \infty$ — with *no* shared or compatible objective among the parts (the hypotheses mention only $S^C, k, L, \mathrm{slack}$). Non-vacuity: `toy_meta_persistence_fires`.

- `regulator_selection` — WP0162 Proposition 1: $\arg\min_R K(R \mid W) = \arg\max_R [M(W{:}R) - K(R)]$.

- `regulator_selection_order` — the order form of the same proposition: under the chain rule, minimizing $K(R \mid W)$ is equivalent to maximizing $M(W{:}R) - K(R)$, as an order-equivalence rather than an argmin identity.

- `AlgorithmicEmergence.no_universal_compressor` / `no_universal_compressor_micro` — WP0007's discovery barrier in the form that carries the intuition: no total computable compressor returns, for every string admitting a description shorter than itself, such a description. Asks nothing about optimality. Consumes `CompressibleUndecidable`, the undecidability of $\{x : K(x) < |x|\}$, which is a *different* classical fact from `KThresholdUndecidable` — a diagonal slice of an undecidable set need not be undecidable.

- `AlgorithmicEmergence.no_compression_improver_of_raw` — WP0007's arbitrary-threshold corollary as the paper states it: derived from `no_universal_compressor` by specializing at $b = |x|$, and consuming *no* undecidability input of its own. `no_compression_improver` proves the same conclusion directly from `KThresholdUndecidable`; that route is sound but its hypothesis is not independent.

- `AlgorithmicEmergence.threshold_undecidable_of_compressible` — why. Undecidability of the threshold relation $\{(x, b) : K(x) < b\}$ *follows* from undecidability of its diagonal slice $\{x : K(x) < |x|\}$ along the computable map $x \mapsto (x, |x|)$. The converse fails, which is why the raw-length statement needs its own proof; once the slice is settled the full relation costs nothing further. So the development rests on one classical input here, not two.

- `AlgorithmicEmergence.compressor_of_threshold` — the hierarchy, recorded as a lemma because it is easy to assert backwards. A threshold procedure specializes at $b = |x|$ to a universal compressor, so "no universal compressor" *implies* "no threshold procedure". The raw-length statement is the stronger one and is **not** a corollary of the arbitrary-b theorem; the converse fails, since a procedure required to work only at $b = |x|$ is a weaker object than one required to work at every b.

- `AlgorithmicEmergence.no_compression_improver` / `no_compression_improver_micro` — WP0007's discovery barrier: no computable procedure returns, for every string (resp. micro-experiment) and threshold b, a description shorter than b whenever one exists. Strictly weaker a demand than optimality, and still impossible: validity already gives $\text{len}(A(x, b)) < b \Rightarrow K(x) < b$, so the threshold guarantee would make the undecidable predicate $K(x) < b$ decidable. The AIT input is `KThresholdUndecidable`. The companion observation — that the predicate is *semidecidable*, so open-ended search finds a shorter description whenever one exists and only fails to halt otherwise — is deliberately not formalized: it would need an operational semantics for programs, a halting-in-t-steps predicate and the dovetailing construction. The theorem rules out a *total* procedure that halts on every input while exploiting every available compression; semidecidability is the complementary positive fact about partial search, so nothing proved here rests on it.

- `AlgorithmicEmergence.no_universal_emergence_constructor` — the abstract emergence transfer of WP0007 drafts up to v0.31. It does *not* cover the accepted edition's proposition on discovery and regret barriers for reusable models (periodic-block counting under the revised definition), which that paper proves at paper level only. On the unrestricted computable-generator class, a total constructor guaranteed to return a qualifying emergent generator whenever one exists decides a set sandwiched between two compressibility classes, and no decidable set separates them. The compression-to-emergence embedding enters as named hypotheses at two raw-length thresholds (`WrapExists` at the tighter $|x| - c$, since wrapping needs headroom for framing overhead and margins; `UnwrapBeats` at the looser $|x|$, since unwrapping leaks overhead; plus `UnwrapValid` and `ConstructorGuarantee`); the AIT input is `SeparatorUndecidable`, the fixed-slack diagonal in separator form. The two-threshold sandwich replaced an earlier same-threshold statement whose paper-level wrapper could not meet the unwrap bound (2026-08-21 repair). The proposition's scope note — restricted classes inherit only when the wrapped generators lie inside them — is carried by the hypotheses, not by the theorem.

- `AlgorithmicEmergence.no_computable_schedule` — WP0007's no-computable-schedule proposition, the schedule refinement of the discovery barrier: no total computable function bounds the halting time of a sound step-indexed search on every positive instance $K(x) < b$. A schedule would decide the threshold predicate — run the search for the scheduled time and report whether anything was returned. The search enters abstractly (`SearchSound`: on negative instances nothing is ever returned, which is K's minimality in step-indexed dress; `IsSchedule`: the promised bound on positive instances), so this formalizes the *negative* companion of semidecidability without building the dovetailer; the AIT input is again `KThresholdUndecidable`.

- `AlgorithmicEmergence.identification_barrier` — WP0007's optimality barrier: no computable procedure returns, for every finite micro-experiment, a shortest program for its macrodata. "Reduction is not construction." The reduction runs along any `Faithful` embedding; WP0007 uses the one-observation identity experiment ν_y, and the shift-and-read register of its residual-information barrier is another instance; the AIT input is the uncomputability of K, assumed as the named hypothesis `KUncomputable`. `identification_barrier_conditional` is the same statement with complexity taken conditional on the microlaw and observation map; it is a variant kept for the development's own sake, and is *not* what WP0007's conditional-form corollary says — that is `conditional_complexity_uncomputable` above. WP0007's numbering has shifted three times as results were inserted ahead of others, so entries here name results rather than number them.

- `AlgorithmicEmergence.no_additive_approximation` — the unconditional form of WP0007's optimality barrier: relaxing "shortest" to "within a fixed additive constant c of shortest" does not restore computability, for any c. The AIT input here is `KNotApproximable`, which is *not* implied by `KUncomputable` as stated — it is the Berry argument run with slack, and follows from `KNotApproximableOnCompressible` below. Assumed here.

- `AlgorithmicEmergence.no_additive_approximation_compressible` — WP0007's optimality barrier in the form the paper states: unbounded additive regret already on *compressible* records. Strictly stronger than the unconditional statement above, and required by the paper's existence/construction tree, in which both computational barriers live on the branch where a compression gap exists. The AIT input is `KNotApproximableOnCompressible`, the padded-witness strengthening of `KNotApproximable`: running the Berry-with-slack diagonal over $\mathrm{pad}(z) = z0^{|z|}$, which is compressible for long z while $K(\mathrm{pad}(z)) \geq K(z) - O(1)$ remains unbounded, yields a compressible witness. Assumed here; the argument is in the declaration's docstring. WP0007 (accepted edition) does not contain it: its optimality barrier is proved from the blind-spot proposition, which applies to valid constructors and uses injectivity, not to an arbitrary computable f. `no_additive_approximation_compressible_finite_horizon` and `no_universal_compressor_finite_horizon` specialize the two barriers to a record-preserving embedding $e \circ \nu$, WP0007's finite-horizon inheritance corollary.

- `AlgorithmicEmergence.no_bounded_regret_emergence_constructor` — part (b) of the same abstract transfer, likewise not part (b) of the accepted WP0007's reusable-model proposition: a total valid scientific-model constructor cannot keep the regret of its *returned model code* below any fixed bound, even on instances that admit a qualifying scientific model. Composes the fixed-overhead compiler (`CompilerOverhead`) with the fixed-slack non-approximability input (`KNotApproximableOnSlackCompressible`), the slack ensuring the bad instance lies on the favorable branch. Part (a) is `no_universal_emergence_constructor` above.

- `AlgorithmicEmergence.relational_optimality_barrier` — WP0007's optimality barrier stated without an optimum: for every margin and slack there is a record carrying an explicit compression witness *and* an explicit competitor the constructor's output loses to by more than the margin. No shortest description appears in statement or proof, so the result survives a reading on which K is not available as a number; classically it implies the numerical regret form, since every valid description is at least K. The analytic inputs are named hypotheses: `LengthUnbounded` (the paper's counting argument — validity forces distinct programs for distinct records, and only finitely many programs are short), `CompetitorBound` and `BoundOutrun` (the competitor is describable from the search index, so its length is $O(\log k)$), and `WitnessCompresses` (the padded program beats the literal length by the declared slack). This declaration is the arithmetic core: it does not say that the two programs print the

record, and it requires compression at every index. The statement matching WP0007's corollary is `relational_optimality_barrier_valid`, which carries the output predicate $U(p) = x$ for both programs into the conclusion and needs compression only from some index on (`WitnessCompressesEventually`), as the paper's estimate gives.

- `AlgorithmicEmergence.counting_bound` / `few_low_complexity_strings` / `most_states_incompressible` — the counting step of WP0007's residual-information barrier, proved outright rather than assumed (the shift-register identity $x(\mu_{n,y}) = y$ and the injective shortest-program map are supplied when it is instantiated): distinct strings need distinct shortest programs, so fewer than 2^m strings have a shortest conditional program of length $< m$, and the complement is therefore more than a $1 - 2^{-d}$ fraction of the state space. The bound holds for every *fixed* conditioning string shared across the ensemble and does not depend on it — no choice of a common microlaw and observation map improves the fraction.

- `AlgorithmicEmergence.conditional_complexity_uncomputable` — WP0007's conditional-form corollary: conditional complexity relative to $(\mathcal{D}_n, \rho_n, n, C)$ is not computable (WP0007 v0.8.0 notation: ρ the projection, C the observer frame). The conditioning *varies with the input*, so this does not follow from uncomputability at a fixed conditioning string; the formalization checks the closing step of the paper's diagonal argument, with the counting witness and the constant-size search as its two named hypotheses.

- `AlgorithmicEmergence.compression_of_good_model` / `sequential_codelength_eq_logloss` — WP0007's bridge from exact description to scientific theory, which is asymmetric. A short program for one record need not be a useful generative model, and that direction is argued in the paper rather than formalized ("useful model" is not a formal predicate). The direction the interpretation rests on is the two-part coding bound: a simple model assigning the data high probability *witnesses* a short description of them, so a predictive macromodel must realize reusable compression and the limits on finding compression bound the substrate every such model needs. The bound is a named hypothesis (classical, Li–Vitányi); the log-loss identity is proved outright, and is the precise sense in which prediction and compression coincide — *sequential probabilistic* compression, not compression in general.

- `CertifiedPrograms.marking_empty_invariant`, `marking_step_invariant`, `marking_step_preserves_events`, `marking_step_covers`, `observed_mono`, `observed_sound`, `observed_eventually_contains`, `marking_run_invariant`, `marking_event_budget`, and `marking_eventual_coverage` — WP0228's finite persistent-marking construction. A round-robin scan of observed certified neighborhoods marks every fresh vertex when their count reaches the threshold h. Each emission consumes at least h fresh vertices, so $Jh \le |\text{marked}| \le M$. Marks retain a related representative, and every final class of size at least h is eventually represented. The eventual-coverage proof uses soundness and completeness of the supplied pair stream, not a decision procedure for the final relation or a completion detector.

- `CertifiedPrograms.fixed_bits_length`, `fixed_bits_value`, `gamma_code_length`, `gamma_length_mono`, `gamma_parse_append`, `gamma_prefix_free`, `descriptor_length`, `descriptor_parse_append`, `descriptor_prefix_free`, `descriptor_rejects_bad_header`, `global_descriptor_length`, and `local_descriptor_length` — actual self-delimiting descriptors with exact length and parsing proofs. The gamma-style code uses the usual unary length header and a least-significant-bit-first residual payload; its length is exactly $\ell_\gamma(a) = 2\lfloor \log_2(a+1) \rfloor + 1$. Two headers followed by the specified index field form a prefix-free family; the parser rejects $t > L$.

- `CertifiedPrograms.words_exact_length`, `words_exact_complete`, `words_exact_card`, `words_`

through_length, words_through_complete, bounded_binary_card, class_log_range, event_count_fits_width, and marking_rank_bound — the finite binary universe contains $2^{L+1} - 1$ strings, and the proved marking budget bounds each event rank by the available global index width. structure_image_card_le shows that a fixed map assigning a structural class to each candidate cannot increase cardinality.

- CertifiedPrograms.whole_function_minimum_exists, whole_function_minimum_le, and whole_function_minimum_congr minimize over all source strings implementing the same whole partial function, including its domain. certificates_to_common_reference composes two certificates to a fixed reference implementation. Certification requires a genuine Computable proof for its stream and explicit semantic soundness. No concrete optimal evaluator is instantiated.

- CertifiedPrograms.combine_decoder_costs and near_minimal_cost_corollary — scalar arithmetic for the two decoder costs and the additive-minimality specialization. These results take their displayed input inequalities as hypotheses. The later WholeFunctionPrograms module supplies actual source decoders, coverage, and their resulting whole-function reconstruction bounds under named uniform compiler laws. WholeFunctionTree supplies the unrestricted count under the standard tree-labeling principle and stream-decoder compilation. A concrete optimal evaluator and its compiler instances remain outside these interfaces; no final multiplicity or information-distance bound is postulated.

- ComputationCores.run_intertwines and experiment_outputs transport every finite experiment through a map that intertwines each admitted operation. observable_conjugacy constructs a state conjugacy from equal initialized behavior for accessible observable deterministic machines; initialized_intertwiner_unique proves uniqueness. These are existence statements, without an effective classifier. overwrite_rigidity forces a map commuting with coordinate overwrites to have the declared coordinate form; a permutation is handled by reindexing. The finite-product reachability argument and the other survey results in WP0229 remain paper proofs.

- exists_functional_core and functional_core_le_implementation — WP0007's task-relative functional-core benchmark. Given a finite realization, a shortest representative exists and its length is at most the acquired implementation cost plus the fixed interface-wrapper cost. The conditional interpreter carries finite task and prior codes; wrapper correctness and length are explicit hypotheses. The definitions functionalCoreComplexity, IsFunctionalCore, and approximateCoreComplexity minimize description length for the declared behavior, exactly or within a fixed discrepancy tolerance. The exact benchmark preserves the model's task outputs, including imperfect predictions, and asserts no optimization of task loss or runtime. Interpreter computability, core extraction, and a new transfer of the discovery barriers are outside these results. The accompanying no-hypercompression inequality remains a classical probability result proved at paper level in WP0007.

- AlgorithmicEmergence.fixed_task_computable_iff_factorization — WP0007's construction–decision equivalence for one fixed coarse-graining and macroquery. A computable constructor followed by a computable correct evaluator makes the selected answer map computable; the composition direction is also exposed as fixed_task_computable_of_factorization. Conversely, every computable answer map factors through the identity constructor. That converse stores the microscopic input as the model and therefore asserts neither compression nor scientific reuse. The fixed-projection consequence is query_slice_computable_of_query_complete, whose contrapositive is no_query_complete_of_noncomputable_slice. These results consume no AIT hypothesis.

- `AlgorithmicEmergence.chaitin_certification_ceiling` — WP0007's classical certification limit (an appendix remark since the 2026-08-18 restructure; the paper claims exactly three barriers and bills this as classical): certified complexity lower bounds are capped at a constant fixed by the theory and the reference machine, so no macromodel longer than that constant is certifiably shortest for any record. Same obstruction as `chaitin_blocks_minimality`, stated for macromodel optimality rather than self-models. Uncomputability is uniform over the family; unprovability is per-instance. Chaitin's bound is the named hypothesis and Lean checks only the closing step; the step from capped lower bounds to uncertifiable shortness is paper-level. `faithful_comp` records that a record-preserving embedding composed with a faithful one is faithful.
- `algorithmic_emergence` — **superseded.** This declaration stated WP0007's former "no algorithm finds the optimal coarse-graining" result. WP0007 v0.3.0 withdrew that theorem: the objective is degenerate under a constant observation map, the target moves as C varies, and no reduction from the unrestricted structure function into the induced subclass was ever given. The declaration is retained only because `corollaryB` documents the regulatory case, where WP0193 fixes the target and the reduction does go through. Cite `identification_barrier` for WP0007.
- `AITProb.probabilistic_regulator_theorem_conditional` — ART sharp form in the $2^{-K(R|W)}$ reading (posterior favors regulators simple given the world).
- `PrefixMachine.lemma1_posterior_bounds` — the Bayes↔Kolmogorov bridge, $\frac{1}{c_2}2^{K(x)-|p|} \le P(p \mid x) \le \frac{1}{c_1}2^{K(x)-|p|}$.
- `self_regulation_temporal_model` — Proposition 3, $K(S_{t+\tau} \mid S_t, E, C) \le K(S_{t+\tau} \mid C) - \Delta_{\text{self}} +$ 2 slack.
- `pers_eq_nmai`, `persistent_pos` — persistence as temporal self-information.

Assumed bridges, and what is deliberately not formalized. `PushPull.lean` assumes one classical control-theory fact — *Nagumo's theorem*, that for a locally Lipschitz field on a closed interval an inward-pointing boundary condition is equivalent to forward invariance — as the predicate `NagumoBridge`. This is the dynamical counterpart of not re-proving the coding theorem. It is an assumed bridge, not a KT result, and nothing in the module derives the sign conditions from it. `ModelOrPay.lean` assumes `MAICapped` and nothing else. The headline conjecture it was written to check — that a faithful regulator sustaining Δ_N either has $\mathrm{I}_K(W{:}R) = \Omega(\Delta_N)$ or dissipates $\Omega(\Delta_N)\, k_B T \ln 2$ — is *not stateable* in `AITFrame`: Q_{diss} is a physical quantity and the frame has no bath, no temperature, and no notion of a physical realization of W or R. The same holds for the escape dichotomy, which quantifies over "any physical realization," and for the saturation claim, which quantifies over sinks of finite capacity. We record the absence as a specification of the work remaining — it needs stochastic thermodynamics over a defined coupling — and not as a result. The known theorem in that shape is Shannon-form ($\langle W_{\text{ext}} \rangle \le -\Delta F + k_B T\, I(W;R)$); whether it admits a single-sequence algorithmic strengthening with I_K in place of I is open.

Satisfiability witnesses: `toy_symmetry` (symmetry of information holds for `Toy`), `toyMachine_coding` (coding theorem holds for `ToyMachine`), `toy_self_model_fires` (the self-model corollary fires with a genuine gap $\Delta_{\text{self}} = 3$). **Guards:** `#check_failure (peers A S)` (part-whole), and `ystar_form_holds` vs `rawy_form_fails` (symmetry must use y^*, not y).

4 Theorem inventory and formalization status

Every numbered/named formal statement across the KT corpus, classified **[STD]** (standard AIT/probability/logic — axiom candidate) or **[KT]** (Kolmogorov-Theory-specific — proof target), with status. Sources: ART paper,[2] Pattern Persistence (WP0162),[3] Agent Know Thyself (WP0192), Regulatory Coarse-Graining (WP0193), world-models paper,[4] and the foundational AIT-of-consciousness paper.[5]

ID (paper)	Statement (abbreviated)	Class	Status
WP0004	Generator preservation induces the unique surjective homomorphism between realized transformation monoids. Future-observation equivalence is the greatest stable content-preserving equivalence; finite refinement stabilizes before the number of states. Common bijective factors conjugate actions. Surjective factors, projected inverses, prepared reset, and the two-sided shift distinguish global reduction from initialized simulation. Integer iteration has the usual quotient by its kernel.	STD	**Proved**
WP0004	The orbit and class-preserver maps form an adjunction, recover equivalences, and identify the fixed subgroups of orbit saturation. Structural extractors and forgetting maps order these comparisons. Generated contextual congruence is adjoint to rule extraction; the contextual kernel is the greatest congruence below an extractor kernel. The cyclic-three and modulo-four examples prove the need for subgroup saturation and contextual closure, including strict recovery failure for a restricted ambient group.	STD	**Proved**
WP0004	A stable finite enumeration is proved equal to the generated transformation monoid. Kernel reduction verifies the four bit transformations, the sixteen-map record/receiver monoid, its twenty-four-map extension, the changed receiver quotient under use bypass, and the counter and Boolean commutator examples. Deterministic matrices preserve composition and distinguish transformations, while the bit example proves that linear extension need not be injective.	STD	**Proved**
WP0004	An explicit mutable-register evaluator verifies six programs computing the fifteenth power. Write-version extraction gives two nonisomorphic marked multiplication graphs. A single bijective register renaming gives exactly four storage classes. The four class-preserving permutation groups have orders one, four, thirty-six, and seven hundred twenty.	STD	**Proved**

ID (paper)	Statement (abbreviated)	Class	Status
WP0004	A residual estimate and a Lipschitz estimate give the error recurrence for trajectories with the same operation schedule. Iteration proves the geometric, nonexpansive, contractive, and varying-constant bounds. Distance is preserved by isometries, while a three-point example shows that a fixed tolerance is not transitive.	STD	**Proved**
WP0004	A transition becomes a permutation by storing previous states and pairing the resulting injective edges. Two involutions give the forward step. Every positive-time state and observation is preserved, the history length records elapsed time, and integer powers act faithfully. The realized cyclic action group is isomorphic to the integers. Primitive recursiveness is preserved in both directions.	STD	**Proved**
WP0004	The history lift preserves the complete partial execution protocol for arbitrary preparation and observation. A fixed primitive-recursive transition runs the bounded partial-recursive-code evaluator and stores its result. Its reversible history extension computes exactly the partial-recursive functions with explicit program preparation and readout. A computable transition with computable preparation and observation has partial-recursive stopping semantics. The ordinary transition is noninjective; the reversible transition and inverse are primitive recursive. No machine-step complexity bound is asserted.	STD	**Proved**
WP0004	An action factors through its faithful realized image. The group quotient removes the action kernel; the monoid quotient removes the kernel congruence. Nonisomorphic cyclic groups of orders four and two realize the same bit transformations. Two faithful order-two actions on four states have different fixed-state counts and admit no action isomorphism, even allowing a group isomorphism. Labeled conjugacy preserving input, completion, and readout transports every finite run.	STD	**Proved**
WP0215	A partial-recursive evaluator generates finite answer tables. Totality is required only at the selected model code. The general conditional-map coding principle gives one description overhead across models, response functions, and prefix lengths.	KT	**Proved**

ID (paper)	Statement (abbreviated)	Class	Status
WP0215	Source streams with computable stages generate a threshold tree. Disjoint contributor sets bound every finite antichain, and every prefix of the total target enters. The standard bounded-width tree-labeling principle and fixed compilation law give the whole-function multiplicity bound. The tree-labeling theorem is a named AIT premise; it is not re-proved here.	KT	**Proved**
WP0215	An executable scan over binary sources and a computable certificate stream produces permanent marks and chronological events. Coverage and event budgets justify the global and local prefix descriptors. Named fixed-compiler laws turn their proved decoding behavior into the certified whole-function reconstruction bound, including partial functions with their domains.	KT	**Proved**
WP0215	Strong lumpability gives a unique stochastic quotient whose evolution commutes with projection for every initial law and every finite iterate. The content kernel is the greatest unrestricted content-preserving equivalence; declared admissible families can have no coarsest or maximal member. The counterexamples concern abstract equivalence families, not physical admissibility.	KT	**Proved**
WP0215	Closure is equivalent to representative invariance and to a nondecreasing co-membership indicator on paired states. Two-sided closure is equivalent to conservation of that indicator and normalizes the fiber-permutation group. Arbitrary equivariant group actions induce closed orbit quotients. The module also proves content factorization, operation composition, and finite iteration.	KT	**Proved**
WP0215	The explicit encoder and parser recover finite tables with exactly $k\lceil \log_2 n \rceil$ bits and conditional prefix-freeness. Together with the supplied orbit presentation, named fixed-interpreter and orbit-decoder laws give the equivariant-update complexity bound. The prefix code is proved; an optimal universal-machine construction remains a standard AIT premise.	KT	**Proved**

ID (paper)	Statement (abbreviated)	Class	Status
WP0215	Conditional coding mass bounds the actual finite family of near-minimal programs. A staged enumeration and parsed deficit/index descriptor recover either source from the other, giving an explicit excess-plus-logarithm bound. Coding and universal simulation are named standard AIT premises; interpreter soundness, completeness, and stability are explicit. The module also proves normalized common-semantics and report-information consequences.	KT	**Proved**
WP0215	The actual clipped heating controller contracts in the temperature/forecast-error norm. Finite-horizon perturbation bounds, affine record recoding, and mismatch identities follow. These are real-algebraic controller results; experiential assignments and physical implementations are outside their statements.	KT	**Proved**
WP0215 AIT interface	Conditional coding, universal prefix-interpreter simulation, and the existing information laws	STD	**Axiom** (named hypotheses)
WP0255	Co-membership lemma: for an onto reduction, conservation of the co-membership indicator is forward closure with an injective reduced update; for a bijective update it is two-sided closure with a bijective reduced update.	KT	**Proved**
WP0255	Two-sided corollary: an action commuting with a bijection commutes with its inverse; the orbit quotient is closed in both directions, the orbit indicator is conserved, and the reduced update is injective.	KT	**Proved**
WP0255	Labels-and-dimensions proposition, part (a), set level: an orbit of a finite group has at most as many points as the group; (c): a commuting update carries fixed points to fixed points. The covering-map and quotient-manifold clauses are cited, not formalized.	KT	**Proved**
WP0255	Two-positions proposition: a symmetry fixing every macrostate induces the identity; a bijection carries fibers onto fibers exactly when it normalizes the relabeling group, and then descends to a map commuting with the reduced update.	KT	**Proved**
WP0255	Trivial-partition proposition, necessity: a transitive group of bijections commuting with the update forces equal return-time sets. The sufficiency construction is not formalized.	KT	**Proved**

Reliable physical decisions — WP0251 (OperationalInformation)

ID (paper)	Statement (abbreviated)	Class	Status
Effective search	Partial-recursive search returns the target table from sound, eventual reports	STD	**Proved** (`recover_partrec`, `recover_correct`)
Replacement bound	$K(A_n \mid n) \le K(a_n \mid n) + c$, conditional on the coding interface and adequate effective observation	KT	**Proved** (`replacement_bound`)
Bounded replacement	Bounded conditional replacement complexity implies task computability; the contrapositive gives unboundedness	KT	**Proved** (bounded/unbounded replacement declarations)
Description budget	A supplied replacement-description budget bounds answer complexity with fixed overhead	KT	**Proved** (`budget_transfer`)
Complete report	Coding and success-weighted surprisal premises imply a bound at any positive reliability margin	KT	**Proved** (`reliable_report_bound`)
Binary event	$p(-\ln q) \le d(p\|q) + h(p) \le d(p\|q) + \ln 2$, including $p = 0, 1$ with $0 < q < 1$	STD	**Proved** (binary-event declarations)
Report composition	Coding and a supplied binary-divergence budget imply the report bound; unit reference probability is separate	STD	**Proved** (binary-budget and reference-one declarations)
Threshold margins	Replacement error $1/12$ and estimation error $1/48$ preserve the separation at $1/2$	STD	**Proved** (estimate declarations)
Coding and Meyer	Standard conditional prefix-map bound and bounded-prefix characterization	STD	**Axiom** (named hypotheses)
Quantum realization	Trace-distance control, rational state net, logarithmic copy budget, and replenishable biased source	STD	**TODO** (paper proofs)
ART paper — The Algorithmic Regulator (Entropy 2026, 28, 257)			
Lemma 1	$P(p \mid x) = 2^{-\lvert p\rvert}/m(x)$; sandwiched by coding thm	STD	**Proved** (`lemma1_...`)
Theorem 2	$\Pr((W,R) \mid x, E) \le C\, 2^{M(W:R)} 2^{-\Delta}$	KT	**Proved** (`prob_reg_thm`)
Theorem 2 (sharp)	retains regulator cost: $\le C\, 2^{M} 2^{-\Delta} 2^{-K(R)}$	KT	**Proved** (`..._sharp`)
Lemma 2	OFF run lower-bounds world: $K(O_{W,\varnothing}) \le K(W) + c_0$	KT	**Axiom** (hyp. `hoff`)
Eq. (6)	wrapper bound post $\le (1/c_1) 2^{K(x) - K(W,R)}$	KT	**Proved** (`wrapper_bound`)
Theorem 1	coupled-pair posterior tilt $\in [\cdot] 2^{K(x) - K(W) - K(R) + M}$	KT	**Proved** (`theorem1_...`)
Theorem 3	$m(O_{W,R})/m(O_{W,\varnothing}) \asymp 2^{\Delta}$ (as-if objective)	KT	**Proved** (`theorem3_...`)

ID (paper)	Statement (abbreviated)	Class	Status
Theorem A4	low-complexity output ⇒ strict tiny shrinkage of $K(W \mid R)$	KT	**Proved** (`low_complexity_shrinkage`)
Defs 1–2	internal model ($M(W{:}R) > 0$); good regulator ($\Delta > 0$)	KT	n/a (encoded)
Thms A1–A3, Lem A1	coding thm (uncond/cond), Kraft counting, excess-length decay	STD	**Axiom**
Pattern Persistence — WP0162			
Proposition 3	self-regulation ⇒ temporal self-model (Eq. 44)	KT	**Proved** (`self_reg_...`)
Proposition 1	regulator selection $\arg\min K(R \mid W) = \arg\max[M - K(R)]$	KT	**Proved** (`regulator_selection`)
Proposition 2	persistence conservation ledger $K(O_W) = \mathrm{I}_K + K(O_W \mid R^*) \pm O(\log)$	KT	**Proved** (`persistence_...`)
Meta-persistence	collective $\mathrm{Pers}_C \geq 1 - (L{+}\text{slack})/k$; parts need not align	KT	**Proved** (`meta_persistence`)
Proposition 4	self-model incompleteness (3 obstructions)	KT	**Proved** (`quine_floor`,...)
PP! ontology	four cells are patterns; agenthood per cell, composition settles nothing	KT	**Proved** (`PP.token_is_agent`, `PP.collective_not_agent_here`,...)
WP0162 §3	telehomeostasis is fixed by the objective alone	KT	**Proved** (`PP.class_is_self_targeting`)
WP0207	agentoptosis gap: agent ∧ not telehomeostatic	KT	**Proved** (`PP.agentoptosis_gap`)
WP0207 v0.8	event requires agenthood + internal selection; no target/beneficiary field	KT	**Proved** (`PP.AgentoptoticEvent`, `PP.other_target_not_self_targeting`)
WP0207 v0.8	target/composition promote nothing; collective telehomeostasis conditional	KT	**Proved** (`PP.targeted_class_not_agent`, `PP.composition_does_not_imply_agenthood`, `PP.CollectiveSelfTargeting`)
WP0207 v0.8	terminal action underdetermines target	KT	**Proved** (`PP.terminal_action_underdetermines_target`)

ID (paper)	Statement (abbreviated)	Class	Status
WP0207 v0.8	one other-directed objective: retains in ordinary state, terminates in exceptional	KT	**Proved** (`PP.ordinary_retains_exceptional_terminates`)
WP0207 v0.8	deletion threshold $0 < pC_{\mathrm{FN}} - (1-p)C_{\mathrm{FP}} \iff p > \vartheta$	KT	**Proved** (`PP.deletion_threshold`)
WP0162 §5	agenthood is a positive Δ_{self}, computed not asserted	KT	**Proved** (`PP.externally_regulated_not_agent`)
WP0162 §4	persistence unfolds to NMAI, inherited from `Persistence`	KT	**Proved** (`PP.persists_iff_nmai`)
Proposition 3 (PP)	every agent has a temporal self-model	KT	**Proved** (`PP.agent_has_temporal_self_model`)
WP0207	agent yet not telehomeostatic	KT	**Proved** (`PP.agent_need_not_be_self_targeting`)
Orbit-label thm	conserved orbit label = "generalized energy"	KT	**Proved** (`genEnergy_conserved`)
Agent Know Thyself — WP0192			
Proposition 1	temporal self-model (= WP0162 Prop. 3)	KT	**Proved** (same)
Principle 1	self-model incompleteness (= WP0162 Prop. 4)	KT	**Proved** (Phase 4)
Conjecture	acquired self-model prior	KT	n/a (conjecture)
From More Is Different to Algorithmic Emergence — WP0007			
Functional core	shortest exact representative exists; its length is bounded by the acquired implementation plus fixed wrapper cost	KT	**Proved**
Core definitions	exact and approximate conditional description minima	KT	n/a (definitions encoded)
No hypercompression	baseline probability of a k-bit coding gain is at most 2^{-k}	STD	n/a (classical result; paper-level proof, not formalized)
Discussion proposition	fixed-task answer computable $\iff$ computable constructor–evaluator factorization; converse stores the input	KT	**Proved** (`fixed_task_computable_iff_factorization`, `no_query_complete_of_noncomputable_slice`)

ID (paper)	Statement (abbreviated)	Class	Status
Regulatory Coarse-Graining — WP0193			
Theorem B	targeted sufficient projection uncomputable	STD	**Proved** (`theoremB`)
Corollary B	regulatory coarse-graining uncomputable	KT	**Proved** (`corollaryB`)
Lemma 1	uncomputability of $h_{x\to y}$	STD	**Axiom** (via V–V)
Proposition 1	existence of a bounded regulation-sufficient projection	KT	**Proved** (`existence`)
Theorem A	selection uncomputability	KT	**Proved** (`theoremA`)
Cor. C1–C3	blanket uncomputability/fuzziness; acquisition; discovery	KT	n/a (interp.)
Inheritance Architecture — WP0058			
Prop. 1(i)	no total, domain-recognizing write-back encoder	KT	**Proved** (`no_universal_decoder`)
— engine	a total such inverse would decide achievability	KT	**Proved** (`achievable_computable_of_inverse`)
Prop. 1(ii)	some computable D admits no such encoder	KT	**Proved** (`exists_map_without_decoder`)
— non-vacuity	some computable D has an undecidable achievable set	STD	**Axiom** (`ExistsUndecidableAchievableSet`)
Prop. 1(iii)	a *partial* recursive inverse exists, with domain exactly $\mathcal{A}$	KT	**Proved** (`partial_inverse_exists`, `dovetail`)
— $\mathcal{A}$ is r.e.	the achievable set of a computable D is recursively enumerable	KT	**Proved** (`achievable_rePred`)
— unbounded in time	the dovetailing search has no bound on its running time	KT	n/a (resource claim; no cost model in this development)
Prop. 2	heritable burden net of background, $K(C\,\|\,B) \le K(H\,\|\,B) + O(\log)$	KT	**Proved** (`decoder_charged_rel`)
— background-free form	$K(C_{\mathcal{D}}) \le K(H) + O(\log)$	KT	**Proved** (`decoder_charged`)
— cond. subadd. rel. B	$K(x\,\|\,B) \le K(x\,\|\,\langle y, B\rangle) + K(y\,\|\,B) + O(\log)$	STD	**Axiom** (`SubadditivityCondRel`)
Channel bound (lemma)	both links: $\mathrm{I}_K(a : H'\,\|\,w) \le \mathrm{I}_K(a : z\,\|\,w) \le K(z\,\|\,w) + O(\log)$	KT	**Proved** (`channel_bound`)
— left link	data processing under the Markov premise $a \to z \to H'$ given w	STD	**Axiom** (`DataProcessingCond`)

ID (paper)	Statement (abbreviated)	Class	Status
— right link	$\mathrm{I}_K(a{:}z\,\|\,w) \le K(z\,\|\,w)+O(\log)$; info via the channel only	KT	**Proved** (`condIK_le_condRight`)
Master bound	$\lambda_B \le K(H'\,\|\,H)+O(\log)$; the acquired state drops out	KT	**Proved** (`bandwidth_le_cond`)
Prop. 3 (Darwinian, λ_B)	write-back channel empty: $\lambda_B = \mathrm{I}_K(a : H'\,\|\,H,\sigma) \le O(\log)$	KT	**Proved** (`no_program_no_write_back`)
Prop. 3 (Darwinian, λ_{sel})	zeroth-order: total $\mathrm{I}_K(a : H'\,\|\,H) \le K(\sigma)$	KT	**Proved** (`darwinian_bandwidth_le_selection`)
— regime from data proc.	`Darwinian` is `DataProcessingCond` at the selection signal	KT	**Proved** (`darwinian_of_dataProcessing`)
Prop. 3 (Lamarckian)	first-order: $\lambda_B \le K(C(a)\,\|\,H)$, the decoder image	KT	**Proved** (`lamarckian_bandwidth_le_decoder_image`)
— joint $\ge$ marginal	$K(x\,\|\,z) \le K(\langle x,y\rangle\,\|\,z)+O(\log)$	STD	**Axiom** (`JointGeMarginal`)
— cond. subadditivity	$K(x) \le K(x\,\|\,y)+K(y)+O(\log)$	STD	**Axiom** (`SubadditivityCond`)
— conditioning $\le$ uncond.	$K(x\,\|\,y) \le K(x)+O(\log)$	STD	**Axiom** (`CondLeUncond`)
Corollary 1	write-back is capped by the write-back program the encoder can form	KT	**Proved** (`trivial_decoder_transmits_nothing`)
Retention bound	$M(n) = \mathrm{I}_K(w : H_{g+n}\,\|\,H_g) \le K(w\,\|\,H_g)+O(\log)$	KT	**Proved** (`retention_le_program`)
Prop. 4(i)	linear box-constrained kernel: the optimum is bang-bang	KT	**Proved** (`bangbang_optimal`)
Prop. 4(ii)	antitone net value $\Rightarrow$ retained set is $[0,L)$	KT	**Proved** (`optimal_kernel_antitone_is_initial_segment`)
— L is first crossing	$q^*(n)=1 \iff n<L$ for `FirstCrossing` L	KT	**Proved** (`qStar_eq_one_iff_lt_first_crossing`, `qStar_eq_zero_of_first_crossing_le`)
— existence of L	a first crossing exists if v is somewhere nonpositive	KT	**Proved** (`exists_firstCrossing`)

ID (paper)	Statement (abbreviated)	Class	Status
— threshold rule	q^* well defined and admissible	KT	**Proved** (`qStar_of_pos`, `qStar_of_nonpos`, `qStar_admissible`)
Prop. 4(iii)	$v = R - \kappa$: optimum retains exactly where $\kappa < R(n)$, i.e. $\tau_{\rho,\theta}$	KT	**Proved** (`persistence_is_first_crossing`, `netValue_antitone`)
Prop. 4(iv)	$\kappa < R_0 e^{-n/\tau} \iff n < \tau \log(R_0/\kappa)$	KT	**Proved** (`exponential_crossing`, `qStar_exponential`)
— proportionality	crossing lag $\propto \tau$ at fixed cost ratio	KT	**Proved** (`exponential_persistence_proportional_to_tau`)
— non-vacuity	an antitone v whose optimum is a nontrivial initial segment	KT	**Proved** (`vTest_antitone`, `vTest_firstCrossing`)
Hypothesis 1	persistence matching, $\lambda_P^* \sim \tau_E$, in real lineages	KT	n/a (conjecture; the *optimization* is proved above)
Push–Pull Motifs — WP0186			
Theorem 1	opposed flows necessary: $H(x_+) \geq \overline{D}$ and $G(x_-) \geq -\underline{D}$	KT	**Proved** (`theorem1_necessity`)
— contrapositive	a single-signed controller cannot hold the band	KT	**Proved** (`single_signed_...`)
— boundary step	the $\forall D$ condition collapses to two worst cases	KT	**Proved** (`inward_iff`)
Nagumo's theorem	inward on $\partial B \Leftrightarrow$ forward invariance	STD	**Axiom** (`NagumoBridge`)
Theorem 2	saturating edge rule renders the band invariant	KT	**Proved** (`theorem2_sufficiency`)
Thms 1+2	sharp form: invariance $\Leftrightarrow$ both edge conditions	KT	**Proved** (`regulation_iff`)
Remark 2	non-strict off-band bound does *not* give attraction (counterexample)	KT	**Proved** (`nonstrict_fails_...`)
— repair	strict margin ε: $\dot{x} \leq -\varepsilon$, finite-time entry	KT	**Proved** (`margin_gives_...`)

ID (paper)	Statement (abbreviated)	Class	Status
Minimality	sign-diverse $\Rightarrow |S| \geq 2$; two flows attain it	KT	**Proved** (`sign_diverse_...`, `minimal_is_two`)
Corollary 1	$-\kappa(x - x^\star)$ is not single-signed; splits into push + pull	KT	**Proved** (`restoring_not_...`, `restoring_decomp...`)
App. Lemma 1	gap identity $\Delta = S \pm O(\log N)$ from (A2)–(A4)	KT	**Proved** (`lemma1_gap_...`)
— (A2)–(A4)	OFF splits as $K(D_{\text{pred}}) + K(\eta)$; ON tracks η	STD	**Axiom** (`OffSplit`, `OnResidual`)
Remark 7	$S = 0 \Rightarrow$ no gap (pure noise buys no model content)	KT	**Proved** (`no_structure_no_gap`)
Prop. 1 (withdrawn)	sustained Δ *forces* $M(W{:}R) \gtrsim \Delta$	KT	n/a (retracted; not entailed)
The ART Tail Does Not Follow — WP0203			
Proposition 1	$\mathrm{I}_K(W{:}R) \leq K(R) + O(\log)$: a cheap regulator cannot model	KT	**Proved** (`simple_regulator_...`, `model_implies_big`)
— clamp	$K(R) = O(1) \Rightarrow \mathrm{I}_K(W{:}R) = O(1)$ for *every* world	KT	**Proved** (`clamp_MAI_bounded_...`)
— MAI cap	$\mathrm{I}_K(x{:}y) \leq K(y) + O(\log)$	STD	**Axiom** (`MAICapped`)
Proposition 2	per-pair 2^{-k} does not lift to a fiber of size 2^k (witness)	KT	**Proved** (`perpair_does_not_lift`, `multiplicity_...`)
Invertible coupling	$\mathrm{I}_K \geq K(R) - K(x_{\text{ON}})$ holds with $\mathrm{I}_K \ll \Delta$ (witness of insufficiency)	KT	**Proved** (`invertible_bound_...`)
— when it reaches Δ	exactly when $K(x_{\text{OFF}}) \leq K(R)$	KT	**Proved** (`invertible_reaches_...`)
"Model it or pay for it"	$\mathrm{I}_K = \Omega(\Delta_N)$ or $Q_{\text{diss}} = \Omega(\Delta_N) k_B T \ln 2$	KT	n/a (conjecture; not stateable in `AITFrame`)
Gap transfer	grounding $\Rightarrow |\Delta - \Delta_Z| \leq 2\varepsilon + O(\text{slack})$	KT	**Proved** (`gap_transfer`)
Grounded inequality	$\Delta_Z \leq \mathrm{I}_K(z_0{:}R) + L_Z \leq \mathrm{I}_K(W{:}R) + L_Z + O(\text{slack})$	KT	**Proved** (`grounded_readout`, `grounded_inequality`)

ID (paper)	Statement (abbreviated)	Class	Status
Repaired corollary	grounded, non-collapsing: $\mathrm{I}_K(W{:}R) \geq \Delta - L_Z - 2\varepsilon - O(\text{slack})$	KT	**Proved** (`repaired_corollary`, `repaired_good_regulator`)
Forced residual	$\mathrm{I}_K \leq KR + \text{slack} \Rightarrow L_Z \geq \Delta_Z - KR - O(\text{slack})$	KT	**Proved** (`simple_regulator_forces_residual`)
Incomplete reconstruction	residual equals record information plus the remaining conditional complexity, within chain-rule slack	KT	**Proved** (`residual_with_missing_information`)
— record bound	retain the missing conditional complexity; no closure premise	KT	**Proved** (`balance_with_missing_information`)
Residual = completion	closure: $\lvert L_Z - \mathrm{cIK}(z_0{:}Q \mid \langle z, R\rangle)\rvert \leq 2\,\text{slack}$	KT	**Proved** (`residual_is_completion`, `grounded_balance`)
Exact balance (v16)	$\Delta_N^{\zeta} = \mathrm{I}_K(\zeta_\varnothing{:}R \mid C) + L_{\mathrm{cf}} - \mathrm{I}_K(\zeta_R{:}R \mid C) - K(\zeta_R \mid \zeta_\varnothing, R, C) \pm O(\text{slack})$	KT	**Proved** (`exact_counterfactual_balance`)
— residual bound	drop regulated-branch terms + data processing $\Rightarrow \Delta_N^{\zeta} \leq \mathrm{I}_K(W{:}R \mid C) + L_{\mathrm{cf}} + O(\text{slack})$	KT	**Proved** (`gart_reguland_from_exact_balance`, `grounded_inequality_from_exact_balance`)
— closed ledger	closure: residual term = $\mathrm{cIK}(\zeta_\varnothing{:}Q \mid \zeta_R, R, C)$ in the equality	KT	**Proved** (`exact_counterfactual_balance_closed`)
Implicit self-model	self-regulation query: $\mathsf{SM}^{\mathrm{imp}} \geq \Delta_Z - L_Z - O(\text{slack})$; grounded: $\geq \Delta - L_Z - 2\varepsilon - O(\text{slack})$	KT	**Proved** (`self_regulation_forces_self_model`, `.._grounded`)
Retained-input reconstruction	world description recoverable from world records; null output computable from it	KT	**Proved** (`closure_by_construction`)
Inverse-state reconstruction	complete final records and a cost-bounded left inverse recover the initial joint state	KT	**Proved** (`initial_from_complete_records`)
Null-output reconstruction	inverse evolution, world decoding, and null replay; costs add	KT	**Proved** (`null_from_complete_records`)

ID (paper)	Statement (abbreviated)	Class	Status
Model compensation (v22)	retained model makes compensation invertible and an exact prediction zero	KT	**Proved** (`compensation_reversible`, `model_cancellation`)
Joint description conservation	supplied inverse and forward/backward map costs	KT	**Proved** (`joint_complexity_preserved`)
Available-model inference	reading the model and generating the null output bound the residual and initial information	KT	**Proved** (`model_residual`, `model_initial_information`)
Class-mass transfer (v24.2)	GART on each explanation puts $\{I \le \Delta - k\}$ inside $\{L \ge k - s\}$; nonnegative weights are monotone	KT	**Proved** (`class_mass_transfer`, `.._tsum`, `residual_bounded_class_mass_zero`)
ART, residual-transfer form	abstract `post` weights at allowance $4s$: mass of low shared information $\le$ mass of large residual; 0 below the residual-bound threshold; interpretation supplied separately	KT	**Proved** (`posterior_residual_transfer`, `posterior_residual_bounded_zero`)
Conditional tail given a residual bound	in ART's original shape: $\Pr[I \le \Delta - k \mid L \le \eta] \le 2^{-(k-\eta-s)}$, and 0 for $k > \eta + s$	KT	**Proved** (`conditional_tail_given_residual`)
ART, exact form	$\log_2 \mathrm{post}(e,x) = -K(R \mid x) - L - H \pm O(\text{slack})$ up to the coding constants; upper half implies the gap form; a selected code's length offset δ is retained in the specified bridge or absorbed only under a uniform bounded-offset premise	KT	**Proved** (`exact_exponent`, `art_exact_form`, `art_exact_upper`, `residual_dominates_gap`)
ART, shared-information form and bridge	$\log_2 \mathrm{post}(e,x) = K(x) - K\langle W,R\rangle - \log_2 c$ with $c \in [c_1, c_2]$ and $K(x) - K\langle W,R\rangle = I_K(W{:}R) - K(R) - \Delta - K(W \mid y_\varnothing) \pm s$; $I_K(W{:}R) = \Delta - L + (K(R) - K(R \mid x)) + (K(W \mid y_\varnothing) - K(W \mid x, R, y_\varnothing)) \pm 5s$ in complexity differences; the symmetric I_K/cIK form is the symmetric bridge with its larger explicit tolerance	KT	**Proved** (`shared_information_exponent`, `bridge_identity`, `four_term_exponent`, `excess_nonneg`)
ART with the residual (two-sided)	$-K(R) - L - K(W \mid y_\varnothing) - 8s - \log_2 c_2 \le \log_2 \mathrm{post}(e,x) \le I_K(W{:}R) - K(R) - \Delta + s - \log_2 c_1$; the endpoints differ by $\varepsilon + H_0 + 9s + \log_2(c_2/c_1)$	KT	**Proved** (`art_with_residual`)
— two-sided chain rules, outputs computable	$K\langle a,b\rangle = Ka + K(b \mid a) \pm s$; conditional form; $K\langle W,R\rangle = K\langle x,R,y_\varnothing,W\rangle \pm s$	STD	**Axiom** (`Chain`, `CondChain`, `OutputsComputable`)

ID (paper)	Statement (abbreviated)	Class	Status
Conditional ART (v26)	positive finite conditioning mass; zero probability below $\Delta - \eta - h$ and an exponential envelope	KT	**Proved** (`conditional_small_residual_zero`, `conditional_deficit_zero`, `conditional_art_tail`)
Unadjusted-threshold guard	arithmetic GART permits $I < \Delta$ with probability one, a fixed residual, and an unbounded gap	KT	**Proved** (`unadjusted_threshold_counter example`)
Countable posterior transfer (v25)	variable gap and allowance; strict-event confidence bound and countable zero case	KT	**Proved** (`class_mass_transfer_variable_tsum`, `class_mass_residual_confidence_tsum`, `residual_bounded_class_mass_zero_tsum`, `gart_mass_transfer_tsum`)
Independent information moment	$\mathbb{E}\, 2^{I_K(W:R)} \le ab$ from marginal coding domination and pair Kraft	STD	**Proved** (`independent_information_moment`)
Independent regulation (v25)	$\Pr(\Delta - L - h \ge t) \le \min\{1, ab2^{-t}\}$; event-wise uniform-allowance consequence	KT	**Proved** (`independent_regulation_probability`, `independent_regulation_threshold`)
Conditional independent regulation (v26)	joint-event bound divided by the positive residual-event probability; probability capped at one	KT	**Proved** (`independent_regulation_given_residual`)
Specified-program bridge	actual length $K(e) + \delta$, common evidence: posterior $= 2^{-\delta}\mathrm{post}(e, x)$	KT	**Proved** (`compatible_program_factor`, `specified_weight_coding_bounds`)
Explicit ART constants	sharp bound with $(1/c_1)2^{c_0-\delta}$; uniform upper constant; chain allowance retained	KT	**Proved** (`sharp_art_specified`, `sharp_art_uniform`, `conditional_art_specified`)

ID (paper)	Statement (abbreviated)	Class	Status
Symmetric-information bridge	difference-form bridge plus pair/context costs: $7s + c_{\text{pair}} + c_{\text{condpair}} + c_{\text{context}}$	KT	**Proved** (`information_difference`, `conditional_information_difference`, `bridge_symmetric_information`)
Selected residual endpoints	length offset shifts both exponents; separation $\varepsilon + H_0 + 9s$; logarithms retain coding constants	KT	**Proved** (`art_with_residual_specified`, `log2_sandwich`)
Local sampling bound	GART only where $L \le \eta$; conditional bound divides by $q_\eta > 0$	KT	**Proved** (`threshold_from_local_gart`, `given_residual_from_local_gart`)
Finite product-event mass	normalized marginals imply each event has mass at most one	STD	**Proved** (`product_event_mass_le_one`)
Evidence-local transfers	strict confidence bound; variable allowance; separately assumed exponential residual tail	KT	**Proved** (`conditional_confidence_transfer`, `conditional_variable_transfer`, `conditional_exponential_transfer`)
— GART instantiation	named chain and data-processing facts supply the pointwise balance	KT	**Proved** (`independent_gart_probability`)
— marginal coding and pair Kraft	computable-law domination by prefix-code weights; total shortest-pair-code weight at most one	STD	**Axiom** (`Coding Dominated`, `PairKraft`, named hypotheses)
— unconditional map cost	description bound for a supplied fixed computable map	STD	**Axiom** (`Description MapBound`, a named hypothesis)
— map cost	conditional description bound for each supplied computable map	STD	**Axiom** (`ConditionalMapBound`, a named hypothesis)
— asymptotic form	$\mathrm{I}_K(W{:}R) \ge (1 - o(1))\Delta$ for $\varepsilon, \lambda = o(\Delta)$	KT	n/a (analysis on top of the finite inequalities)

ID (paper)	Statement (abbreviated)	Class	Status
Horizon conditioning (v32.10)	$\mathrm{I}_K(W{:}R) \geq \mathrm{I}_K(W{:}R \mid N) - K(N) - O(1)$; the bound at every horizon and its supremum over a finite family	KT	**Proved** (`conditioning_on_horizon`, `horizon_bound_every`, `horizon_bound_sup`)
Descriptor bound and descriptive saving (v34.3)	$\mathrm{I}_K(W{:}R) \geq \max\{K(m), \Delta - L\} - O(1)$; $\Delta - L - O(1) \leq K(y_\varnothing) - K(y_\varnothing \mid R) \leq \mathrm{I}_K(W{:}R) + O(1)$, and the same upper bound for any readout	KT	**Proved** (`key_bound`, `intervention_and_gap`, `readout_saving_upper`, `descriptive_saving`)
Shared-content split (v34.3)	$\mathrm{I}_K(W{:}R) = \mathrm{I}_K(y_\varnothing{:}R) + \mathrm{I}_K(W{:}R \mid y_\varnothing) \pm 3s$ under null-output computability	KT	**Proved** (`shared_content_split`)
— self-information, second-argument data processing, chain through a computable output	$K(m) \leq \mathrm{I}_K(m{:}m) + s$; $\mathrm{I}_K(m{:}m) \leq \mathrm{I}_K(m{:}R) + s$; $K(a) = K(y_\varnothing) + K(a \mid y_\varnothing) \pm s$	STD	**Axiom** (`SelfInformation`, `DataProcessingSnd`, `ChainThroughOutput`, `ChainPair`)
Algorithmic Persistence Balance — roadmap v2 (PersistenceFlow)			
APB with error	$\nu \leq \sum_5 J + \delta + 2s$; δ independent of s	KT	**Proved** (`algorithmic_persistence_balance_with_error`)
Forces flow with error	$J_a + J_\Xi \geq \nu - \sum d - \delta - 5s$	KT	**Proved** (`bounded_persistence_forces_flow_with_error`)
Pigeonhole with error	$2\max\{J_a, J_\Xi\} \geq \nu - \sum d - \delta - 5s$	KT	**Proved** (`flow_pigeonhole_with_error`)
Closure within slack	original APB and flow bounds specialize to $\delta = s$	KT	**Proved** (`algorithmic_persistence_balance`, `bounded_persistence_forces_flow`, `flow_pigeonhole`)
Self-code overload	recoverability forces $K(Z_1 \mid Z_0, C) \geq \nu - \delta - s$	KT	**Proved** (`self_code_overload`)

ID (paper)	Statement (abbreviated)	Class	Status
Conditional update capacity	$K(Z_1\|Z_0, C) \le K_0$ implies $\nu \le K_0 + \delta + s$	KT	**Proved** (`bounded_self_code_update_capacity`)
Unconditional self-code capacity	$K(Z_1) \le K_0$ and conditioning monotonicity give the same bound	KT	**Proved** (`bounded_self_code_capacity`)
No positive gap	witness: NMAI $= 1$ persistence, $\Delta < 0$ raw gap	KT	**Proved** (`persistence_does_not_imply_gap`)
— rate corollary	asymptotic $\nu_P \le r_Z + r_M + r_\sigma + r_a + r_\Xi$	KT	n/a (analysis on top of the finite forms)
— identity-grounded bridge	restricted persistence-to-GART score	KT	n/a (readout definition open; roadmap §3 Cor. 2 is existing GART re-instantiated)
Reconstruction and capacity with neutral names — WP0218 revision (CompletionLedger)			
Grouped completion balance	$K(D\|B) \le K(S\|B) + \mathrm{I}_K(D{:}E\|S,B) + \delta + 2s$	KT	**Proved** (`grouped_completion_balance`)
Grouped: internal budget forces external information	$\mathrm{I}_K(D{:}E\|S,B) \ge K(D\|B) - b - \delta - 2s$	KT	**Proved** (`grouped_bounded_internal_forces_external`)
Grouped: internal capacity	$K(D\|S,B) \le \delta$, $K(S\|B) \le b$ give $K(D\|B) \le b + \delta + s$	KT	**Proved** (`grouped_internal_capacity`)
Ordered completion balance	$K(D\|B) \le \sum_i J_i + \delta + 2s$ along a declared order	KT	**Proved** (`ordered_completion_balance`)
Ordered: k internal budgets force external information	$\sum_{\text{ext}} J \ge K(D\|B) - \sum_{\text{int}} d - \delta - (2+k)s$	KT	**Proved** (`ordered_bounded_internal_forces_external`; list lemmas `orderedInfo_append`, `orderedInfo_le_orderedCost`)

ID (paper)	Statement (abbreviated)	Class	Status
Ordered: two external records	$2\max\{J_{E_1}, J_{E_2}\} \geq$ the same right-hand side	KT	**Proved** (`ordered_two_external_pigeonhole`)
Five-record APB is the ordered instance	$[Z^+, \mathcal{W}^+, \sigma, a, \Xi]$, $k = 3$, $(2+3)s = 5s$	KT	**Proved** (`flowSplit5_iff_orderedSplit`, `orderedCost_internal_three`, `apb_flow_from_ordered`)
Ordered split, derived by telescoping	$\mathrm{I}_K(D{:}Q\|B) \leq \sum_i J_i + h_{\log} + h_{\mathrm{conv}} + n\,s$ from the easy chain rule at each step (`ChainAlong`), the hard chain rule on the joint record (`JointChainLower`), and the nested/tuple context conversion (`NestedConversion`)	KT	**Proved** (`orderedInfo_telescope`, `orderedSplit_telescoped`)
Ordered balance with the split derived	$K(D\|B) \leq \sum_i J_i + \delta + h_{\log} + h_{\mathrm{conv}} + (n+1)s$	KT	**Proved** (`ordered_completion_balance_telescoped`)
State graphs and permutations — WP0218 Appendix E (StateGraph)			
Singleton support is a permutation graph	unique successor and predecessor at every state $\iff R(i,j) \iff \sigma(i) = j$ for a permutation σ; empty space included	KT	**Proved** (`permGraph_of_unique`, `unique_of_permGraph`, `unique_iff_permGraph`)
Exact-step layers	k exact steps $\iff \sigma^k$; every future and past layer is a singleton	KT	**Proved** (`steps_iff_pow`, `steps_unique_succ`, `steps_unique_pred`)
Conserved orbit label from singleton support	the permutation's `genEnergy` is invariant along every allowed transition	KT	**Proved** (`orbit_label_of_unique`)
Compressive models and innovation capacity — WP0218 (ModelAccounting)			
Lossless recoding	Mutual reconstruction preserves conditional complexity within $\varepsilon + s$	STD	**Proved** (`lossless_recode_complexity`)
Acquired model cost	Model, parameters, and residual contribute ordered costs, plus $\delta + 3s$	KT	**Proved** (`model_parameter_residual_bound`)
Innovation cost	Recoverable parameter plus $n-c$ incompressible conditional bits requires $\kappa+n-c-\varepsilon-2s$	KT	**Proved** (`innovation_lower_bound`)

ID (paper)	Statement (abbreviated)	Class	Status
Complementary retention	Internal budget $\kappa + b$ leaves at least $n - c - b - \varepsilon - \delta - 4s$ in the complement	KT	**Proved** (`innovations_force_complement`)
Internal retention	Internal recovery implies $n \leq b+\delta+c+\varepsilon+3s$	KT	**Proved** (`innovations_limit_internal_retention`)
Repeated-block records	Recover the residual, the parameter at positive horizon, and the record; raw length $n(q+1)$	KT	**Proved** (`block_residual_recovery`, `block_parameter_recovery`, `block_record_recovery`, `block_stream_length`)
Algorithmic localization dynamics — WP0218 (Localization, TokenTransport)			
Sum identity	$L_A+I_K+L_W = K(A,W)$, exact on abstract coordinates	KT	**Proved** (`localization_sum_exact`)
Delta identities	coordinate-sum change $= \Delta K(A,W)$; $\Delta K_A + \Delta K_W - \Delta I_K = \Delta K(A,W)$	KT	**Proved** (`localization_delta_exact`, `marginal_delta_exact`)
Exact conservation	fixed joint budget $\Rightarrow$ conserved coordinate sum	KT	**Proved** (`localization_conserved_exact`)
Reversible invariance	mutual ε-reconstruction: $\lvert Ky - Kx\rvert \leq \varepsilon +$ slack	STD	**Proved** (`reversible_complexity_invariance`)
ALB (reversible)	localization sum moves $\leq \varepsilon +$ slack under fixed partition	KT	**Proved** (`localization_balance_reversible`)
— cond. data processing	$\mathrm{I}_K(D{:}Y) \leq \mathrm{I}_K(X{:}Y) + K(D\lvert X) +$ slack	STD	**Axiom** (`CondDataProcessing`)
Overlap lemma	$\mathrm{I}_K(X{:}Y) \geq KD - K(D\lvert X) - K(D\lvert Y) -$ 2 slack	KT	**Proved** (`recoverable_description_overlap`)
— data-processing step derived	$\mathrm{I}_K(D{:}Y) \leq \mathrm{I}_K(X{:}Y)+K(D\lvert X)+h+4$ slack from starred symmetry (both forms), recovery from X^*, subadditivity through D, and plain-to-starred conversion with log allowance h	STD	**Proved** (`cond_data_processing_from_steps`)

ID (paper)	Statement (abbreviated)	Class	Status
Overlap lemma from the proof steps	$\mathrm{I}_K(X{:}Y) \geq KD - K(D\|X) - K(D\|Y) - h -$ 5 slack, h explicit	KT	**Proved** (`recoverable_description_overlap_steps`)
Recurrent recoverability from the proof steps	$\mathrm{I}_K(S_t{:}S_{t+\tau}) \geq KD - \varepsilon_t - \varepsilon_{t+\tau} - h - 5$ slack	KT	**Proved** (`recurrent_recoverability_steps`)
Recurrent recoverability	ε-recoverable D_P at two times bounds cross-time I_K, Pers	KT	**Proved** (`recurrent_recoverability_implies_mai_bound`, `.._persistence_bound`)
Gates involutive	$C_{A\to W}$, $C_{W\to A}$, SWAP, and mode gates self-inverse	KT	**Proved** (`cnotAW_involutive`, `cnotWA_involutive`, `swap_involutive`, `gateFor_involutive`)
Mode reachability	one gate converts any canonical mode into any other	KT	**Proved** (`canonical_mode_reachable`)
Transport plans exist	equal-total profiles admit a 3×3 plan (northwest corner)	KT	**Proved** (`transport_plan_exists`)
Plans realizable	disjoint token-pair gates realize every entry, reversibly	KT	**Proved** (`transport_plan_realizable`, `applyGates_involutive`)
Minimum moved mass	$N - \sum_i \min(p_i, p'_i) = \frac{1}{2}\|p - p'\|_1$ (doubled `Nat` form)	KT	**Proved** (`diag_le_min`, `exists_min_moved_plan`, `minimum_moved_mass`)
— canonical realizability	incompressible-block representative of any profile	KT	n/a (standard AIT existence argument; manuscript Prop.)
— token–profile bridge	$O(\log N)$ translation from token lengths to K coordinates	KT	n/a (manuscript-level; deliberately not formalized)
Computation Cores — WP0229			

ID (paper)	Statement (abbreviated)	Class	Status
Composite words	`run_intertwines`: transport finite compositions	STD	**Proved**
Readouts	`experiment_outputs`: preserve outputs of composite experiments	STD	**Proved**
Observable models	`observable_conjugacy`: equal behavior with accessibility and observability gives a conjugacy	STD	**Proved**
Uniqueness	`initialized_intertwiner_unique`: uniqueness on accessible states	STD	**Proved**
Overwrites	`overwrite_rigidity`: commutation forces coordinate form	STD	**Proved**
Near-Minimal Equivalent Programs — WP0228			
Persistent marking	executable finite scan, charging budget and eventual coverage	STD	**Proved** (`marking_...`)
Descriptor codes	exact lengths, correct parsing and prefix-free record formats	STD	**Proved** (`descriptor...`)
Finite counting	binary universe, dyadic threshold and global event width	STD	**Proved** (`bounded_...`)
Structural images	fixed image cardinality cannot exceed candidate cardinality	STD	**Proved** (`structure...`)
Whole-function minimum	existence, comparison and invariance under full semantic equality	STD	**Proved** (`whole_...`)
Integer algebra	combines supplied decoder inequalities; specializes additive excess	STD	**Proved** (`combine_...`)
AIT bound	certified reconstruction and unrestricted count under named standard compilation and tree-labeling premises	STD	**Proved** (explicit AIT interfaces)
The Simulation Is Phenomenally Itself — WP0215			
Orbit-table reconstruction	one image per orbit recovers every equivariant update	STD	**Proved** (`orbit_table_recovery`, `orbit_table_injective`)
Equivariant-law count	at most n^k updates on n states with k represented orbits	STD	**Proved** (`equivariant_update_count`)
Orbit closure	the represented orbit map commutes with the update and its finite iterates	STD	**Proved** (`quotient_update_commutes`, `quotient_update_iterates`)
Conditional table cost	$K(F \mid D) \le k\ell + c_{\text{code}} + c_{\text{dec}}$	STD	**Proved** (`equivariant_update_complexity`)
— coding premises	finite-table code bound and decoder overhead	STD	**Axiom** (named `TableCodingBound`, `OrbitDecoderBound`)

ID (paper)	Statement (abbreviated)	Class	Status
— concrete compiler	optimal prefix machine and effective presentation discovery	STD	n/a (finite-code instantiation is a paper argument)
Common semantics	$\mathrm{I}_K(p{:}q) \geq K(F) - c_1 - c_2 - 4$ slack when F is cheaply recoverable from both	KT	**Proved** (`common_semantics_bound`)
— cost-free form	same, with the frame's own $K(F \mid p^*), K(F \mid q^*)$	KT	**Proved** (`common_semantics_bound_raw`)
— conditioned on D	$\mathrm{I}_K(p{:}q \mid D) \geq K(T_D(f) \mid D) - O(\log n)$	KT	**TODO** (needs conditional forms of both hypotheses)
— data processing	$\mathrm{I}_K(z{:}y) \leq \mathrm{I}_K(x{:}y) + K(z \mid x^*) + O(\log)$	STD	**Axiom** (`DataProcessingIK`)
— self-information	$K(x) \leq \mathrm{I}_K(x{:}x) + O(\log)$	STD	**Axiom** (`SelfInfo`)
— not structure	bound attained while $K(p \mid q^*) = 97$: shared semantics, no shared form	KT	**Proved** (`common_semantics_does_not_give_structure`)
— equal size is not it	equal K, zero I_K, in a frame satisfying all three laws	KT	**Proved** (`equal_complexity_is_not_shared_information`)
Equivalence hierarchy	internal isomorphism on reachable states $\Rightarrow$ equal boundary	KT	**Proved** (`boundary_eq_of_iso`)
— converse false	mod-4 and mod-2 parity: same trace, no isomorphism	KT	**Proved** (`parity4_boundary_eq_parity2`, `parity4_not_internal_iso`)
— no dead code	every state of the four-state machine is reachable	KT	**Proved** (`parity4_all_reachable`)
Finite unfolding	any recurrent machine, any horizon T: an acyclic depth-T evaluator agrees	KT	**Proved** (`finite_unfolding`)
— shape of it	T layers, each a copy of the state; no asymptotic claim	KT	**Proved** (`unfolding_layers_eq_state`)
— horizon bound	a depth-T evaluator reads only the first T inputs	KT	**Proved** (`depth_bounded_ignores_late_input`, `evalHorizon_ignores_late_input`)

ID (paper)	Statement (abbreviated)	Class	Status
— no fixed cover	one evaluator of depth T already fails at horizon $T+1$	KT	**Proved** (`no_fixed_unfolding_covers_all_horizons`)
— uniform version	one acyclic machine, all horizons	KT	n/a (unwritable: `Layered X Y T` is indexed by T)
— unfolding size	growth of the unfolded family with T	KT	n/a (resource claim; no cost model in this development)
Realization relation	$P \models_{\rho,C} \mathfrak{S}$ with commuting and counterfactual conditions	KT	**TODO** (not yet formalized)
Structured experience	$\mathcal{S}(P) \cong \mathfrak{S}(P)$, experience as implemented structure	KT	n/a (not a proof target; no formal referent for experience)
Arithmetic program extraction	value and derivative from an explicit syntax transformation; unique generator in a declared interface	STD	**Proved** (`symbolic_generator_sound`, `generator_unique`)
— declared state/auxiliary interface	conditional coordinatewise recovery limit and unique vector field	STD	**Proved** (`recovery_with_auxiliary_inputs`, `recovered_vector_field_unique`)
— scalar decay	Euler and RK4 expression trees have the same first jet	STD	**Proved** (`decay_common_core`)
World models / Lie groups — Entropy 2025 (separate geometry track)			
Thms A1–A5	Lie transitivity, homogeneous space, moduli stack	STD	n/a (out of scope v1)
Noether (gen.)	continuous symmetry ⇒ conserved quantity (abstract flow core)	KT	**Proved** (`trajLabel_conserved`)
Homogeneous space (transitive action ⇒ G/H)	—	STD	**Proved** (`homogeneousSpace`)
Lie pseudogroups / moduli stacks / Mostow / dim formula	—	geom	**TODO** (not in Mathlib)
Foundational — nix019 / K-input			
Hyp. 1–2, Cons. 1–2	simplicity selection; structured experience; high MAI	KT	n/a (informal)

Legend. **Proved**: machine-checked in Lean, no `sorry`. **Axiom**: assumed (standard result, or a named hypothesis consumed by a corollary). **TODO**: a KT proof target not yet formalized. n/a: definition, conjecture, interpretive remark, or informal claim (not a proof target in the current scope).

5 Roadmap

Priorities are ordered to maximize KT-correctness assurance per unit effort, exploiting the fact that the AIT/Bayes layer (§2) may be assumed.

Phase 1 — close the ART chain. DONE.
The wrapper bound Eq. (6) is now a *derived* theorem (`wrapper_bound`) from the coding theorem (`CodingLB`); ART **Theorem 1** (`theorem1_posterior_tilt`) and **Theorem 3** (`theorem3_onoff_evidence`, log-free form) are proved; `probabilistic_regulator_theorem` now consumes `wrapper_bound` and rests on the coding theorem + Lemma 2. All four `#print axioms` = Lean core only.

Phase 2 — persistence conservation (WP0162 Prop. 2). DONE.
Ledger $K(O_W) = \mathrm{I}_K(O_W{:}R) + K(O_W \mid R^*)$ (`persistence_conservation`, from symmetry of information) and the trade-off "max I_K = min conditional residual" (`conservation_tradeoff`).

Phase 3 — regulator selection (WP0162 Prop. 1). DONE.
$\arg\min_R K(R \mid W) = \arg\max_R [M(W{:}R) - K(R)]$ proved (`regulator_selection`) from the chain rule; the ART sharp form's $2^{-K(R|W)}$ reading is `probabilistic_regulator_theorem_conditional`.

Phase 4 — self-model incompleteness. DONE.
The three independent obstructions: the quine floor (`quine_floor`, $|Q_A| \geq K(A)$), the self-prediction dichotomy (`self_prediction_dichotomy`, an axiom-free diagonalization), and the Chaitin ceiling on minimality (`chaitin_blocks_minimality`).

Phase 5 — coarse-graining uncomputability (WP0193). DONE.
Theorem B and Corollary B (regulatory coarse-graining uncomputable) proved by an abstract reduction to Vereshchagin–Vitányi (`theoremB`, `corollaryB`).

All five roadmap phases are complete. The WP0162 / WP0192 / WP0193 KT corollaries and the full probabilistic ART layer are machine-checked, `sorry`-free, resting only on Lean core plus the named AIT axiom layer of §2.

Later — geometry track.
The Lie-group/Noether layer of the world-models paper is a distinct development (differential geometry in Mathlib), deferred until the AIT/KT core is complete.

Each phase keeps the invariant: KT results `sorry`-free; `#print axioms` = Lean core + named hypotheses; new hypotheses discharged by an extended toy model.

6 Conclusion

We have a green, machine-checked Level-1/1.5 core: the probabilistic ART (Theorem 2), the Bayes↔Kolmogorov bridge (Lemma 1), the temporal self-model (Proposition 3), and persistence, all proved from an explicit, satisfiable AIT/Bayes axiom layer, with the KT ontology typed so the whole-vs-part error cannot compile. This document is the living status tracker; the roadmap above closes the remaining KT corollaries paper-by-paper while keeping the soundness invariants

intact.

References

[1] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, CADE 2021; The mathlib Community, *The Lean Mathematical Library*, CPP 2020.

[2] G. Ruffini et al., *An Algorithmic-Information-Theoretic Regulator Theorem*, Entropy **28**(3):257, 2026.

[3] G. Ruffini, *Pattern Persistence* (BCOM WP0162), 2026.

[4] G. Ruffini et al., *World models, Lie groups, and symmetry in Kolmogorov Theory*, Entropy **27**(1):90, 2025.

[5] G. Ruffini, *An algorithmic information theory of consciousness*, Neuroscience of Consciousness **2017**(1):nix019.

[6] M. Li and P. Vitányi, *An Introduction to Kolmogorov Complexity and Its Applications*, 4th ed., Springer, 2019.